

Assignment 3 - Computer vision (PG) Trimester 2 2025

Deep Learning for Perception Tasks

This assignment contains 2 parts. The first part (Q1 and Q2) probes your understanding of deep learning for classification. The second part requires you to write a short proposal about a Computer Vision application. You will need to submit two separate PDF files, one for Q1 and Q2, and another for Q3.

Question 1: A simple classifier (40 points)

For this exercise, we provide demo code showing how to train a network on a small dataset called Fashion-MNIST. Please run through the "tutorial style" section of code to get a sense of what it is doing. Then use the code alongside lecture notes and other resources to understand how to use pytorch libraries to implement, train and use a neural network.

For the Fashion-MNIST dataset the labels from 0-9 correspond to various clothing classes so you might find it convenient to create a python list of class names as follows:

```
class_names = ['T-shirt/top', 'Trouser', 'Pullover', 'Dress',  
               'Coat', 'Sandal', 'Shirt', 'Sneaker', 'Bag', 'Ankle boot']
```

You are asked to answer various questions about the neural network classification model, e.g., how it works and your observation of experiments. We'd like you to make modifications to model structure and training scheme, aiming at improving the model performance.

Please make your own text cells and code cells to organise the answer to each question below.

Detailed requirements:

Q1.1 (2 points)

Extract 3 images of different types of clothing from the training dataset, print out the size (shape) of the training images, and display their corresponding labels.

Q1.2 (4 points)

Run the training code for 10 epochs, with different learning rate values. **Fill in the table below and plot the loss curves** for each learning rate.

As a couple of helpful suggestions:

- You may want to plot the loss curves of different learning rates in one figure to facilitate a visually intuitive comparison.
- You may want to plot both training and test curves (either in the same figure or different figures) to better understand/describe the fitting/overfitting of the model.

These suggestions are also useful for considering your answer in other questions below.

Lr	Accuracy
1	
0.1	
0.01	
0.001	

Table 1: Accuracy comparison for different learning rates.

Q1.3 (6 points)

Report the number of epochs when the network converges (or number of epochs for the best accuracy, if it fails to converge). **Fill in the table below and plot the loss curve** for each choice of learning rate. Please run the code for >10 epochs (e.g., 50 or 100) and report the accuracy and the epoch number when you observe convergence. You may want to first clearly define the convergence criteria when answering this question.

Lr	Accuracy	Epoch
1		
0.1		
0.01		
0.001		

Table 2: Accuracy comparison for different learning rates (when converged).

Q1.4 (4 points)

Compare the results in Table 1 and Table 2, what is your observation and your understanding of utility and effect of the learning rate?

Q1.5 (10 points)

Build a wider network by modifying the code that constructs the network so that the hidden layer(s) contain more processing units. Please record the accuracy corresponding with the number of trainable parameters in your model.

Then modify the **original** network to be deeper instead of wider (i.e. by adding more hidden layers). Please record your accuracy against the changed network parameter size.

Plot the loss curve for each experiment. Please also plot the test accuracy and loss for both the wider and deeper architectures and discuss what you observe. Write down your conclusions toward the variation the network structure?

As a helpful advice, to answer this type of question, usually you'd want to fix certain training parameters (e.g., your learning rate) to be identical to ensure minimal interference from them.

Structures	Accuracy	Parameters
Base		
Deeper		
Wider		

Table 3: Accuracy comparison for different model sizes and structure changes.

Q1.6 (4 points)

Calculate the "mean of the gradient norm (a.k.a. gradient magnitude)". You may choose to compute:

1. the mean of the gradient norm to all trainable parameters in your model; or,
2. the mean of the gradient norm to the trainable parameters in a specific layer of your choice (please explain your choice).

Please plot a curve of the "mean of the gradient norm" for 100 training steps (iterations) sampled at a fix interval throughout your training. For example, if your training takes 5000 iterations over 50 epochs, that's 100 iterations per epoch, you should record the "mean of gradient norm" roughly every 50 iterations. Note that gradients will be computed and saved with the trainable weight parameters automatically after you call `loss.backwards()`. Please report the trend/change of the mean of gradient norm and your understanding of why the trend happened.

For more explanation of Q1.6, you could refer to the following example: [compute gradient example](#).

Q1.7 (10 points)

Modify the network structure and training/test to use a small Convolutional Neural Network (CNN) instead of an MLP. Discuss your findings with regard to convergence, accuracy and number of model parameters, in relation to MLPs.

Hint: Look at the structure of the CNN in the Workshop 3 examples.

Question 2: Poking into the Blackbox (20 points)

Q2.1 (8 points)

Experiment with different activation functions (ReLU, Tanh, Sigmoid) and analyse their impact on training performance.

Q2.2 (4 points)

In particular, focus your analysis on the Sigmoid activation function and discuss your finding of training with and without using **Xavier** initialisation method. You may use the provided code for Xavier initialisation for this.

Q2.3 (4 points)

Additionally, plot both the gradient and loss curves for your experiments. For gradient analysis, you may select one representative layer to monitor throughout training and briefly explain your choice.

Q2.4 (4 points)

Discuss how gradients and loss behave across the network for different activation functions and initialisation methods if you observe any difference.

Question 3: Proposal for Practical Applications (40 points)

In this part of the assignment you need to write a report about an application of a computer vision algorithm or technique. This can either be an application that you are aware of and possibly even use, such as a phone app, or it could be speculation -- an application that you think would benefit from using computer vision.

Begin by choosing a particular CV idea, method or problem area, such as:

- removing noise in an image
- increasing the resolution of an image
- detecting and/or identifying objects in an image
- segmenting images into constituents parts
- estimating the depth of an object from one or more images
- estimating the motion of two objects in different frames
- others

Now think about various ways your chosen technique could be used. Here is a list of possible applications you could consider, but you are not restricted to this list, and there will be credit given for sensible invention outside this list (but no penalty if you don't want to be "inventive"):

- image editing systems in your phone;
- enhancement of images from old film;

- obstacle detection and avoidance for a domestic robot;
- facial recognition for phone security;
- cancer detection;
- person tracking and re-identification in security cameras;
- sport decision review systems;
- road-sign detection and interpretation for self-driving cars.

This exercise is a bit different to real world problem-solving paradigm where you have a problem then try to figure out a method to solve it. Here you are aware of what those CV methods can do and try to think of use cases (applications).

In this part, you are required to write a short report (800 words maximum) in which you do the following:

1. Clearly define the CV problem/area and describe its application scenarios.
2. Briefly describe a solution based on image processing, computer vision and/or machine learning.
3. Discuss the advantages and the limitations of this method in various application scenarios.
4. It is important that you will define a set of useful metrics to evaluate the performance of your method and discuss its trade-off specific to the problem you have chosen.
5. You are welcome to cite existing work and take inspiration from literature addressing the problem you choose.

An image is worth thousand words. You may use diagrams, flow charts or other figures in your report to aid the explanation of your solution.

For this question, you do not need to implement your solution; just write the proposal/report and submit it as a separate PDF.

The "tutorial style" example code starts here, reuse parts as you like while writing your report

```
In [1]: import numpy as np # This is for mathematical operations

# this is used in plotting
import matplotlib.pyplot as plt
import time
import pylab as pl
from IPython import display

%matplotlib inline

%load_ext autoreload
%autoreload 2
%reload_ext autoreload
```

```
In [2]: ##### Tutorial Code
#####PyTorch has two primitives to work with data: torch.utils.data.DataLoader and
#####Dataset stores samples and their corresponding labels, and DataLoader wraps
import torch
from torch import nn
from torch.utils.data import DataLoader
from torchvision import datasets
from torchvision.transforms import ToTensor, Lambda, Compose
import matplotlib.pyplot as plt

# Download training data from open datasets.
##Every TorchVision Dataset includes two arguments:
##transform and target_transform to modify the samples and labels respectively.

training_data = datasets.FashionMNIST(
    root="data",
    train=True,
    download=True,
    transform=ToTensor(),
)

# Download test data from open datasets.
test_data = datasets.FashionMNIST(
    root="data",
    train=False,
    download=True,
    transform=ToTensor(),
)
```

We pass the Dataset as an argument to [DataLoader](#). This conveniently wraps an iterable over our dataset and supports automatic batching, sampling, shuffling, and multiprocessing data loading. Here we define a batch size of 64, i.e. each element in the dataloader iterable will return a batch of 64 features and labels.

```
In [4]: batch_size = 64

# Create data loaders.
train_dataloader = DataLoader(training_data, batch_size=batch_size)
test_dataloader = DataLoader(test_data, batch_size=batch_size)

for X, y in test_dataloader:
    print("Shape of X [N, C, H, W]: ", X.shape)
    print("Shape of y: ", y.shape, y.dtype)
    break
```

```
Shape of X [N, C, H, W]: torch.Size([64, 1, 28, 28])
Shape of y: torch.Size([64]) torch.int64
```

Add in a code cell to inspect the training data, as per Q1.1. Each element of the training_data structure has a grayscale image (which you can use `plt.imshow(img[0,:,:])` to display, just like you did in previous assignments.

```
In [5]: #Q1.1

class_names = ['T-shirt/top', 'Trouser', 'Pullover', 'Dress', 'Coat',
               'Sandal', 'Shirt', 'Sneaker', 'Bag', 'Ankle boot']
```

```

image1, label1 = training_data[9]
image2, label2 = training_data[6]
image3, label3 = training_data[2]

print("Image 1 shape:", image1.shape)
print("Label 1:", class_names[label1])

print("Image 2 shape:", image2.shape)
print("Label 2:", class_names[label2])

print("Image 3 shape:", image3.shape)
print("Label 3:", class_names[label3])

fig, axs = plt.subplots(1, 3, figsize=(10, 3))
axs[0].imshow(image1.squeeze(), cmap='gray')
axs[0].set_title(class_names[label1])
axs[0].axis('off')

axs[1].imshow(image2.squeeze(), cmap='gray')
axs[1].set_title(class_names[label2])
axs[1].axis('off')

axs[2].imshow(image3.squeeze(), cmap='gray')
axs[2].set_title(class_names[label3])
axs[2].axis('off')

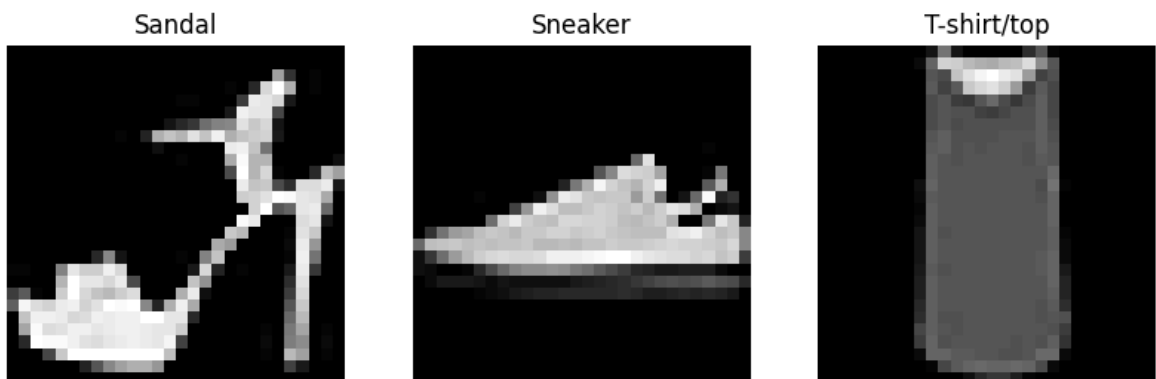
```

```

Image 1 shape: torch.Size([1, 28, 28])
Label 1: Sandal
Image 2 shape: torch.Size([1, 28, 28])
Label 2: Sneaker
Image 3 shape: torch.Size([1, 28, 28])
Label 3: T-shirt/top

```

```
Out[5]: (np.float64(-0.5), np.float64(27.5), np.float64(27.5), np.float64(-0.5))
```



To define a neural network in PyTorch, we create a class that inherits from `nn.Module`. We define the layers of the network in the `init` function and specify how data will pass through the network in the `forward` function. To accelerate operations in the neural network, we move it to the GPU if available.

```

In [6]: # Get cpu or gpu device for training.
device = "cuda" if torch.cuda.is_available() else "cpu"
print("Using {} device".format(device))

import torch.nn.functional as F

# Define model

```

```

class NeuralNetwork(nn.Module):
    def __init__(self):
        super(NeuralNetwork, self).__init__()
        self.flatten = nn.Flatten()
        self.linear_relu_stack = nn.Sequential(
            nn.Linear(28*28, 512),
            nn.ReLU(),
            nn.Linear(512, 512),
            nn.ReLU(),
            nn.Linear(512, 10)
        )

    def forward(self, x):
        x = self.flatten(x)
        logits = self.linear_relu_stack(x)

        return logits

model = NeuralNetwork().to(device)
print(model)

```

Using cpu device

```

NeuralNetwork(
  (flatten): Flatten(start_dim=1, end_dim=-1)
  (linear_relu_stack): Sequential(
    (0): Linear(in_features=784, out_features=512, bias=True)
    (1): ReLU()
    (2): Linear(in_features=512, out_features=512, bias=True)
    (3): ReLU()
    (4): Linear(in_features=512, out_features=10, bias=True)
  )
)

```

```

In [7]: ###Define the loss function and the optimizer
loss_fn = nn.CrossEntropyLoss()
optimizer = torch.optim.SGD(model.parameters(), lr=1e-3)

```

```

In [8]: def evaluate_accuracy(model, dataloader):
    model.eval()
    correct = 0
    total = 0

    with torch.no_grad():
        for X, y in dataloader:
            X, y = X.to(device), y.to(device)
            outputs = model(X)
            _, predicted = torch.max(outputs, 1)
            correct += (predicted == y).sum().item()
            total += y.size(0)

    accuracy = 100 * correct / total
    return accuracy

```

```

In [9]: def train_and_evaluate(learning_rate):
    model = NeuralNetwork().to(device)
    optimizer = torch.optim.SGD(model.parameters(), lr=learning_rate)
    loss_fn = nn.CrossEntropyLoss()

    num_epochs = 10
    losses = []

```



```
for epoch in range(num_epochs):
    model.train()
    total_loss = 0

    for X, y in train_dataloader:
        X, y = X.to(device), y.to(device)
        pred = model(X)
        loss = loss_fn(pred, y)

        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

        total_loss += loss.item()

    avg_loss = total_loss / len(train_dataloader)
    losses.append(avg_loss)
    print(f"[lr={learning_rate}] Epoch {epoch+1}, Loss: {avg_loss:.4f}")

test_accuracy = evaluate_accuracy(model, test_dataloader)

return losses, test_accuracy
```

```
In [10]: learning_rates = ([1, 0.1, 0.01, 0.001])
         results = {}

         for lr in learning_rates:
             print(f"Training with learning rate = {lr}")
             losses, acc = train_and_evaluate(lr)
             results[lr] = {"losses": losses, "accuracy": acc}
             print(f"Test Accuracy for lr={lr}: {acc:.2f}%")
```

```

Training with learning rate = 1
[lr=1] Epoch 1, Loss: 2.2652
[lr=1] Epoch 2, Loss: 2.1873
[lr=1] Epoch 3, Loss: 1.7731
[lr=1] Epoch 4, Loss: 2.0837
[lr=1] Epoch 5, Loss: 2.2906
[lr=1] Epoch 6, Loss: 2.3063
[lr=1] Epoch 7, Loss: 2.3061
[lr=1] Epoch 8, Loss: 2.3061
[lr=1] Epoch 9, Loss: 2.3061
[lr=1] Epoch 10, Loss: 2.3061
Test Accuracy for lr=1: 10.00%
Training with learning rate = 0.1
[lr=0.1] Epoch 1, Loss: 0.6555
[lr=0.1] Epoch 2, Loss: 0.4240
[lr=0.1] Epoch 3, Loss: 0.3747
[lr=0.1] Epoch 4, Loss: 0.3444
[lr=0.1] Epoch 5, Loss: 0.3222
[lr=0.1] Epoch 6, Loss: 0.3043
[lr=0.1] Epoch 7, Loss: 0.2889
[lr=0.1] Epoch 8, Loss: 0.2753
[lr=0.1] Epoch 9, Loss: 0.2633
[lr=0.1] Epoch 10, Loss: 0.2519
Test Accuracy for lr=0.1: 87.41%
Training with learning rate = 0.01
[lr=0.01] Epoch 1, Loss: 1.2827
[lr=0.01] Epoch 2, Loss: 0.6844
[lr=0.01] Epoch 3, Loss: 0.5727
[lr=0.01] Epoch 4, Loss: 0.5177
[lr=0.01] Epoch 5, Loss: 0.4869
[lr=0.01] Epoch 6, Loss: 0.4661
[lr=0.01] Epoch 7, Loss: 0.4503
[lr=0.01] Epoch 8, Loss: 0.4374
[lr=0.01] Epoch 9, Loss: 0.4263
[lr=0.01] Epoch 10, Loss: 0.4166
Test Accuracy for lr=0.01: 83.37%
Training with learning rate = 0.001
[lr=0.001] Epoch 1, Loss: 2.2420
[lr=0.001] Epoch 2, Loss: 2.0617
[lr=0.001] Epoch 3, Loss: 1.7325
[lr=0.001] Epoch 4, Loss: 1.3808
[lr=0.001] Epoch 5, Loss: 1.1530
[lr=0.001] Epoch 6, Loss: 1.0163
[lr=0.001] Epoch 7, Loss: 0.9283
[lr=0.001] Epoch 8, Loss: 0.8675
[lr=0.001] Epoch 9, Loss: 0.8227
[lr=0.001] Epoch 10, Loss: 0.7880
Test Accuracy for lr=0.001: 70.34%

```

```

In [41]: print("| Lr      | Accuracy |")
print("|-----|-----|")
print("| 1      | 10.00%  |")
print("| 0.1    | 87.41%  |")
print("| 0.01   | 83.37%  |")
print("| 0.001  | 70.34%  |")
print("\nTable 1: Accuracy comparison for different learning rates.")

```

Lr	Accuracy
1	10.00%
0.1	87.41%
0.01	83.37%
0.001	70.34%

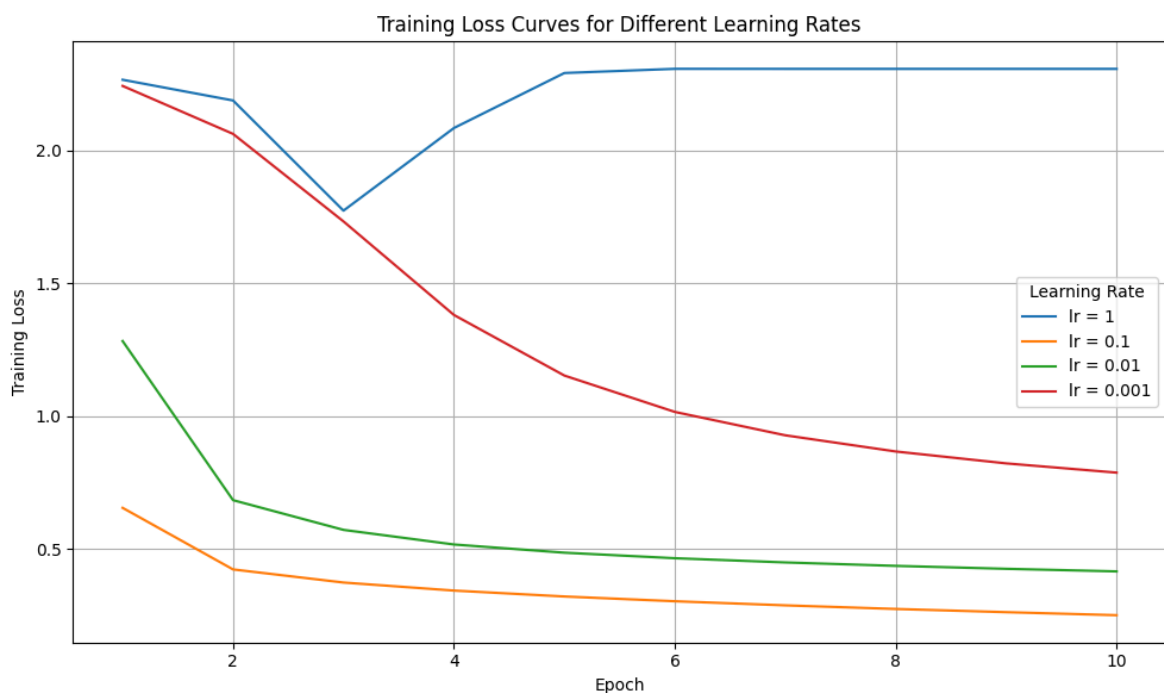
Table 1: Accuracy comparison for different learning rates.

```
In [12]: import matplotlib.pyplot as plt

plt.figure(figsize=(10, 6))

for lr in learning_rates:
    losses = results[lr]["losses"]
    plt.plot(range(1, len(losses) + 1), losses, label=f"lr = {lr}")

plt.title("Training Loss Curves for Different Learning Rates")
plt.xlabel("Epoch")
plt.ylabel("Training Loss")
plt.legend(title="Learning Rate")
plt.grid(True)
plt.tight_layout()
plt.show()
```



```
In [13]: def train_and_evaluate(learning_rate, num_epochs=50):
    model = NeuralNetwork().to(device)
    optimizer = torch.optim.SGD(model.parameters(), lr=learning_rate)
    loss_fn = nn.CrossEntropyLoss()

    train_losses = []
    test_accuracies = []

    for epoch in range(num_epochs):
        model.train()
        total_loss = 0

        for X, y in train_dataloader:
            X, y = X.to(device), y.to(device)
```

```

        pred = model(X)
        loss = loss_fn(pred, y)

        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

        total_loss += loss.item()

    avg_loss = total_loss / len(train_dataloader)
    train_losses.append(avg_loss)

    acc = evaluate_accuracy(model, test_dataloader)
    test_accuracies.append(acc)

    print(f"[lr={learning_rate}] Epoch {epoch+1}/{num_epochs}, Train Loss: {
return train_losses, test_accuracies

```

```

In [14]: learning_rates = [1, 0.1, 0.01, 0.001]
         results = {}

         for lr in learning_rates:
             print(f"\nTraining with learning rate = {lr}")
             train_losses, test_accuracies = train_and_evaluate(lr, num_epochs=50)
             results[lr] = {
                 "train_losses": train_losses,
                 "test_accuracies": test_accuracies,
             }

         best_acc = max(test_accuracies)
         best_epoch = test_accuracies.index(best_acc) + 1
         print(f"Best test accuracy for lr={lr}: {best_acc:.2f}% at epoch {best_epoch}

```

Training with learning rate = 1

```
[lr=1] Epoch 1/50, Train Loss: 2.3929, Test Acc: 10.00%
[lr=1] Epoch 2/50, Train Loss: 2.2983, Test Acc: 10.00%
[lr=1] Epoch 3/50, Train Loss: 2.3244, Test Acc: 10.00%
[lr=1] Epoch 4/50, Train Loss: 2.3061, Test Acc: 10.00%
[lr=1] Epoch 5/50, Train Loss: 2.3061, Test Acc: 10.00%
[lr=1] Epoch 6/50, Train Loss: 2.3061, Test Acc: 10.00%
[lr=1] Epoch 7/50, Train Loss: 2.3061, Test Acc: 10.00%
[lr=1] Epoch 8/50, Train Loss: 2.3068, Test Acc: 10.00%
[lr=1] Epoch 9/50, Train Loss: 2.3015, Test Acc: 10.00%
[lr=1] Epoch 10/50, Train Loss: 2.3061, Test Acc: 10.00%
[lr=1] Epoch 11/50, Train Loss: 2.3061, Test Acc: 10.00%
[lr=1] Epoch 12/50, Train Loss: 2.3061, Test Acc: 10.00%
[lr=1] Epoch 13/50, Train Loss: 2.3061, Test Acc: 10.00%
[lr=1] Epoch 14/50, Train Loss: 2.3061, Test Acc: 10.00%
[lr=1] Epoch 15/50, Train Loss: 2.3061, Test Acc: 10.00%
[lr=1] Epoch 16/50, Train Loss: 2.3061, Test Acc: 10.00%
[lr=1] Epoch 17/50, Train Loss: 2.3061, Test Acc: 10.00%
[lr=1] Epoch 18/50, Train Loss: 2.3061, Test Acc: 10.00%
[lr=1] Epoch 19/50, Train Loss: 2.3061, Test Acc: 10.00%
[lr=1] Epoch 20/50, Train Loss: 2.3061, Test Acc: 10.00%
[lr=1] Epoch 21/50, Train Loss: 2.3061, Test Acc: 10.00%
[lr=1] Epoch 22/50, Train Loss: 2.3061, Test Acc: 10.00%
[lr=1] Epoch 23/50, Train Loss: 2.3061, Test Acc: 10.00%
[lr=1] Epoch 24/50, Train Loss: 2.3061, Test Acc: 10.00%
[lr=1] Epoch 25/50, Train Loss: 2.3061, Test Acc: 10.00%
[lr=1] Epoch 26/50, Train Loss: 2.3061, Test Acc: 10.00%
[lr=1] Epoch 27/50, Train Loss: 2.3061, Test Acc: 10.00%
[lr=1] Epoch 28/50, Train Loss: 2.3061, Test Acc: 10.00%
[lr=1] Epoch 29/50, Train Loss: 2.3061, Test Acc: 10.00%
[lr=1] Epoch 30/50, Train Loss: 2.3061, Test Acc: 10.00%
[lr=1] Epoch 31/50, Train Loss: 2.3061, Test Acc: 10.00%
[lr=1] Epoch 32/50, Train Loss: 2.3061, Test Acc: 10.00%
[lr=1] Epoch 33/50, Train Loss: 2.3061, Test Acc: 10.00%
[lr=1] Epoch 34/50, Train Loss: 2.3061, Test Acc: 10.00%
[lr=1] Epoch 35/50, Train Loss: 2.3061, Test Acc: 10.00%
[lr=1] Epoch 36/50, Train Loss: 2.3061, Test Acc: 10.00%
[lr=1] Epoch 37/50, Train Loss: 2.3061, Test Acc: 10.00%
[lr=1] Epoch 38/50, Train Loss: 2.3061, Test Acc: 10.00%
[lr=1] Epoch 39/50, Train Loss: 2.3061, Test Acc: 10.00%
[lr=1] Epoch 40/50, Train Loss: 2.3061, Test Acc: 10.00%
[lr=1] Epoch 41/50, Train Loss: 2.3061, Test Acc: 10.00%
[lr=1] Epoch 42/50, Train Loss: 2.3061, Test Acc: 10.00%
[lr=1] Epoch 43/50, Train Loss: 2.3061, Test Acc: 10.00%
[lr=1] Epoch 44/50, Train Loss: 2.3061, Test Acc: 10.00%
[lr=1] Epoch 45/50, Train Loss: 2.3061, Test Acc: 10.00%
[lr=1] Epoch 46/50, Train Loss: 2.3061, Test Acc: 10.00%
[lr=1] Epoch 47/50, Train Loss: 2.3061, Test Acc: 10.00%
[lr=1] Epoch 48/50, Train Loss: 2.3061, Test Acc: 10.00%
[lr=1] Epoch 49/50, Train Loss: 2.3061, Test Acc: 10.00%
[lr=1] Epoch 50/50, Train Loss: 2.3061, Test Acc: 10.00%
Best test accuracy for lr=1: 10.00% at epoch 1
```

Training with learning rate = 0.1

```
[lr=0.1] Epoch 1/50, Train Loss: 0.6574, Test Acc: 78.95%
[lr=0.1] Epoch 2/50, Train Loss: 0.4245, Test Acc: 82.16%
[lr=0.1] Epoch 3/50, Train Loss: 0.3751, Test Acc: 83.45%
[lr=0.1] Epoch 4/50, Train Loss: 0.3452, Test Acc: 84.97%
[lr=0.1] Epoch 5/50, Train Loss: 0.3232, Test Acc: 85.58%
[lr=0.1] Epoch 6/50, Train Loss: 0.3053, Test Acc: 86.14%
```

```
[lr=0.1] Epoch 7/50, Train Loss: 0.2898, Test Acc: 86.22%
[lr=0.1] Epoch 8/50, Train Loss: 0.2763, Test Acc: 86.45%
[lr=0.1] Epoch 9/50, Train Loss: 0.2639, Test Acc: 87.29%
[lr=0.1] Epoch 10/50, Train Loss: 0.2527, Test Acc: 87.40%
[lr=0.1] Epoch 11/50, Train Loss: 0.2425, Test Acc: 87.72%
[lr=0.1] Epoch 12/50, Train Loss: 0.2326, Test Acc: 87.73%
[lr=0.1] Epoch 13/50, Train Loss: 0.2233, Test Acc: 87.90%
[lr=0.1] Epoch 14/50, Train Loss: 0.2144, Test Acc: 87.87%
[lr=0.1] Epoch 15/50, Train Loss: 0.2063, Test Acc: 87.86%
[lr=0.1] Epoch 16/50, Train Loss: 0.1981, Test Acc: 88.36%
[lr=0.1] Epoch 17/50, Train Loss: 0.1900, Test Acc: 88.10%
[lr=0.1] Epoch 18/50, Train Loss: 0.1828, Test Acc: 88.02%
[lr=0.1] Epoch 19/50, Train Loss: 0.1757, Test Acc: 87.85%
[lr=0.1] Epoch 20/50, Train Loss: 0.1682, Test Acc: 88.55%
[lr=0.1] Epoch 21/50, Train Loss: 0.1615, Test Acc: 88.40%
[lr=0.1] Epoch 22/50, Train Loss: 0.1543, Test Acc: 88.21%
[lr=0.1] Epoch 23/50, Train Loss: 0.1481, Test Acc: 88.42%
[lr=0.1] Epoch 24/50, Train Loss: 0.1417, Test Acc: 88.17%
[lr=0.1] Epoch 25/50, Train Loss: 0.1353, Test Acc: 87.74%
[lr=0.1] Epoch 26/50, Train Loss: 0.1290, Test Acc: 88.50%
[lr=0.1] Epoch 27/50, Train Loss: 0.1240, Test Acc: 88.53%
[lr=0.1] Epoch 28/50, Train Loss: 0.1181, Test Acc: 88.26%
[lr=0.1] Epoch 29/50, Train Loss: 0.1129, Test Acc: 88.49%
[lr=0.1] Epoch 30/50, Train Loss: 0.1071, Test Acc: 88.69%
[lr=0.1] Epoch 31/50, Train Loss: 0.1030, Test Acc: 88.59%
[lr=0.1] Epoch 32/50, Train Loss: 0.1178, Test Acc: 88.09%
[lr=0.1] Epoch 33/50, Train Loss: 0.1098, Test Acc: 88.83%
[lr=0.1] Epoch 34/50, Train Loss: 0.0932, Test Acc: 88.25%
[lr=0.1] Epoch 35/50, Train Loss: 0.0893, Test Acc: 88.41%
[lr=0.1] Epoch 36/50, Train Loss: 0.0855, Test Acc: 88.53%
[lr=0.1] Epoch 37/50, Train Loss: 0.0826, Test Acc: 88.07%
[lr=0.1] Epoch 38/50, Train Loss: 0.0762, Test Acc: 87.36%
[lr=0.1] Epoch 39/50, Train Loss: 0.0776, Test Acc: 88.31%
[lr=0.1] Epoch 40/50, Train Loss: 0.0728, Test Acc: 88.47%
[lr=0.1] Epoch 41/50, Train Loss: 0.0701, Test Acc: 87.58%
[lr=0.1] Epoch 42/50, Train Loss: 0.0671, Test Acc: 88.45%
[lr=0.1] Epoch 43/50, Train Loss: 0.0648, Test Acc: 88.09%
[lr=0.1] Epoch 44/50, Train Loss: 0.0606, Test Acc: 88.66%
[lr=0.1] Epoch 45/50, Train Loss: 0.0686, Test Acc: 88.49%
[lr=0.1] Epoch 46/50, Train Loss: 0.0643, Test Acc: 88.32%
[lr=0.1] Epoch 47/50, Train Loss: 0.0624, Test Acc: 88.41%
[lr=0.1] Epoch 48/50, Train Loss: 0.0539, Test Acc: 88.58%
[lr=0.1] Epoch 49/50, Train Loss: 0.0532, Test Acc: 88.72%
[lr=0.1] Epoch 50/50, Train Loss: 0.0526, Test Acc: 88.76%
Best test accuracy for lr=0.1: 88.83% at epoch 33
```

Training with learning rate = 0.01

```
[lr=0.01] Epoch 1/50, Train Loss: 1.2907, Test Acc: 71.98%
[lr=0.01] Epoch 2/50, Train Loss: 0.6795, Test Acc: 77.94%
[lr=0.01] Epoch 3/50, Train Loss: 0.5719, Test Acc: 79.66%
[lr=0.01] Epoch 4/50, Train Loss: 0.5185, Test Acc: 80.39%
[lr=0.01] Epoch 5/50, Train Loss: 0.4872, Test Acc: 80.98%
[lr=0.01] Epoch 6/50, Train Loss: 0.4661, Test Acc: 81.46%
[lr=0.01] Epoch 7/50, Train Loss: 0.4501, Test Acc: 81.97%
[lr=0.01] Epoch 8/50, Train Loss: 0.4371, Test Acc: 82.42%
[lr=0.01] Epoch 9/50, Train Loss: 0.4260, Test Acc: 82.83%
[lr=0.01] Epoch 10/50, Train Loss: 0.4163, Test Acc: 83.22%
[lr=0.01] Epoch 11/50, Train Loss: 0.4075, Test Acc: 83.59%
[lr=0.01] Epoch 12/50, Train Loss: 0.3995, Test Acc: 83.92%
[lr=0.01] Epoch 13/50, Train Loss: 0.3921, Test Acc: 84.26%
```

```

[lr=0.01] Epoch 14/50, Train Loss: 0.3852, Test Acc: 84.38%
[lr=0.01] Epoch 15/50, Train Loss: 0.3787, Test Acc: 84.62%
[lr=0.01] Epoch 16/50, Train Loss: 0.3726, Test Acc: 84.81%
[lr=0.01] Epoch 17/50, Train Loss: 0.3669, Test Acc: 85.02%
[lr=0.01] Epoch 18/50, Train Loss: 0.3615, Test Acc: 85.23%
[lr=0.01] Epoch 19/50, Train Loss: 0.3563, Test Acc: 85.40%
[lr=0.01] Epoch 20/50, Train Loss: 0.3514, Test Acc: 85.59%
[lr=0.01] Epoch 21/50, Train Loss: 0.3466, Test Acc: 85.65%
[lr=0.01] Epoch 22/50, Train Loss: 0.3419, Test Acc: 85.76%
[lr=0.01] Epoch 23/50, Train Loss: 0.3375, Test Acc: 85.90%
[lr=0.01] Epoch 24/50, Train Loss: 0.3331, Test Acc: 85.99%
[lr=0.01] Epoch 25/50, Train Loss: 0.3289, Test Acc: 86.09%
[lr=0.01] Epoch 26/50, Train Loss: 0.3247, Test Acc: 86.23%
[lr=0.01] Epoch 27/50, Train Loss: 0.3207, Test Acc: 86.31%
[lr=0.01] Epoch 28/50, Train Loss: 0.3169, Test Acc: 86.39%
[lr=0.01] Epoch 29/50, Train Loss: 0.3130, Test Acc: 86.57%
[lr=0.01] Epoch 30/50, Train Loss: 0.3093, Test Acc: 86.62%
[lr=0.01] Epoch 31/50, Train Loss: 0.3057, Test Acc: 86.78%
[lr=0.01] Epoch 32/50, Train Loss: 0.3021, Test Acc: 86.88%
[lr=0.01] Epoch 33/50, Train Loss: 0.2986, Test Acc: 86.99%
[lr=0.01] Epoch 34/50, Train Loss: 0.2953, Test Acc: 87.06%
[lr=0.01] Epoch 35/50, Train Loss: 0.2920, Test Acc: 87.24%
[lr=0.01] Epoch 36/50, Train Loss: 0.2888, Test Acc: 87.25%
[lr=0.01] Epoch 37/50, Train Loss: 0.2856, Test Acc: 87.31%
[lr=0.01] Epoch 38/50, Train Loss: 0.2826, Test Acc: 87.35%
[lr=0.01] Epoch 39/50, Train Loss: 0.2795, Test Acc: 87.41%
[lr=0.01] Epoch 40/50, Train Loss: 0.2766, Test Acc: 87.48%
[lr=0.01] Epoch 41/50, Train Loss: 0.2737, Test Acc: 87.49%
[lr=0.01] Epoch 42/50, Train Loss: 0.2708, Test Acc: 87.40%
[lr=0.01] Epoch 43/50, Train Loss: 0.2681, Test Acc: 87.39%
[lr=0.01] Epoch 44/50, Train Loss: 0.2653, Test Acc: 87.36%
[lr=0.01] Epoch 45/50, Train Loss: 0.2627, Test Acc: 87.28%
[lr=0.01] Epoch 46/50, Train Loss: 0.2601, Test Acc: 87.32%
[lr=0.01] Epoch 47/50, Train Loss: 0.2574, Test Acc: 87.38%
[lr=0.01] Epoch 48/50, Train Loss: 0.2549, Test Acc: 87.33%
[lr=0.01] Epoch 49/50, Train Loss: 0.2524, Test Acc: 87.41%
[lr=0.01] Epoch 50/50, Train Loss: 0.2499, Test Acc: 87.41%
Best test accuracy for lr=0.01: 87.49% at epoch 41

```

Training with learning rate = 0.001

```

[lr=0.001] Epoch 1/50, Train Loss: 2.2359, Test Acc: 49.38%
[lr=0.001] Epoch 2/50, Train Loss: 2.0463, Test Acc: 50.59%
[lr=0.001] Epoch 3/50, Train Loss: 1.7086, Test Acc: 58.93%
[lr=0.001] Epoch 4/50, Train Loss: 1.3729, Test Acc: 63.13%
[lr=0.001] Epoch 5/50, Train Loss: 1.1532, Test Acc: 64.86%
[lr=0.001] Epoch 6/50, Train Loss: 1.0154, Test Acc: 66.39%
[lr=0.001] Epoch 7/50, Train Loss: 0.9243, Test Acc: 67.72%
[lr=0.001] Epoch 8/50, Train Loss: 0.8607, Test Acc: 69.11%
[lr=0.001] Epoch 9/50, Train Loss: 0.8137, Test Acc: 70.52%
[lr=0.001] Epoch 10/50, Train Loss: 0.7773, Test Acc: 71.94%
[lr=0.001] Epoch 11/50, Train Loss: 0.7479, Test Acc: 73.08%
[lr=0.001] Epoch 12/50, Train Loss: 0.7230, Test Acc: 74.02%
[lr=0.001] Epoch 13/50, Train Loss: 0.7014, Test Acc: 74.83%
[lr=0.001] Epoch 14/50, Train Loss: 0.6821, Test Acc: 75.70%
[lr=0.001] Epoch 15/50, Train Loss: 0.6647, Test Acc: 76.29%
[lr=0.001] Epoch 16/50, Train Loss: 0.6488, Test Acc: 76.95%
[lr=0.001] Epoch 17/50, Train Loss: 0.6343, Test Acc: 77.52%
[lr=0.001] Epoch 18/50, Train Loss: 0.6209, Test Acc: 78.02%
[lr=0.001] Epoch 19/50, Train Loss: 0.6086, Test Acc: 78.48%
[lr=0.001] Epoch 20/50, Train Loss: 0.5973, Test Acc: 78.75%

```

```

[lr=0.001] Epoch 21/50, Train Loss: 0.5869, Test Acc: 79.09%
[lr=0.001] Epoch 22/50, Train Loss: 0.5773, Test Acc: 79.36%
[lr=0.001] Epoch 23/50, Train Loss: 0.5685, Test Acc: 79.68%
[lr=0.001] Epoch 24/50, Train Loss: 0.5603, Test Acc: 79.94%
[lr=0.001] Epoch 25/50, Train Loss: 0.5527, Test Acc: 80.23%
[lr=0.001] Epoch 26/50, Train Loss: 0.5457, Test Acc: 80.45%
[lr=0.001] Epoch 27/50, Train Loss: 0.5391, Test Acc: 80.73%
[lr=0.001] Epoch 28/50, Train Loss: 0.5330, Test Acc: 80.94%
[lr=0.001] Epoch 29/50, Train Loss: 0.5273, Test Acc: 81.05%
[lr=0.001] Epoch 30/50, Train Loss: 0.5220, Test Acc: 81.13%
[lr=0.001] Epoch 31/50, Train Loss: 0.5170, Test Acc: 81.26%
[lr=0.001] Epoch 32/50, Train Loss: 0.5123, Test Acc: 81.34%
[lr=0.001] Epoch 33/50, Train Loss: 0.5078, Test Acc: 81.40%
[lr=0.001] Epoch 34/50, Train Loss: 0.5037, Test Acc: 81.50%
[lr=0.001] Epoch 35/50, Train Loss: 0.4997, Test Acc: 81.55%
[lr=0.001] Epoch 36/50, Train Loss: 0.4960, Test Acc: 81.68%
[lr=0.001] Epoch 37/50, Train Loss: 0.4924, Test Acc: 81.73%
[lr=0.001] Epoch 38/50, Train Loss: 0.4891, Test Acc: 81.84%
[lr=0.001] Epoch 39/50, Train Loss: 0.4859, Test Acc: 81.97%
[lr=0.001] Epoch 40/50, Train Loss: 0.4828, Test Acc: 82.06%
[lr=0.001] Epoch 41/50, Train Loss: 0.4799, Test Acc: 82.01%
[lr=0.001] Epoch 42/50, Train Loss: 0.4771, Test Acc: 82.14%
[lr=0.001] Epoch 43/50, Train Loss: 0.4745, Test Acc: 82.20%
[lr=0.001] Epoch 44/50, Train Loss: 0.4719, Test Acc: 82.23%
[lr=0.001] Epoch 45/50, Train Loss: 0.4694, Test Acc: 82.33%
[lr=0.001] Epoch 46/50, Train Loss: 0.4671, Test Acc: 82.46%
[lr=0.001] Epoch 47/50, Train Loss: 0.4648, Test Acc: 82.54%
[lr=0.001] Epoch 48/50, Train Loss: 0.4626, Test Acc: 82.63%
[lr=0.001] Epoch 49/50, Train Loss: 0.4604, Test Acc: 82.66%
[lr=0.001] Epoch 50/50, Train Loss: 0.4583, Test Acc: 82.70%
Best test accuracy for lr=0.001: 82.70% at epoch 50

```

convergence: the point at which accuracy in training no longer increases very much, that is when there is an improvement in accuracy less than 0.1% after consecutive 5 iterations in the training process. Instead, convergence is Broadway also referred to when the loss value of a training model reduces by a value, less than 0.001 during the same interval of time.

```

In [42]: print("| Lr      | Accuracy | Epoch |")
print("|-----|-----|-----|")
print("| 1      | 10.00%  | 10    |")
print("| 0.1    | 88.83%  | 17    |")
print("| 0.01   | 87.49%  | 36    |")
print("| 0.001  | 82.70%  | 50    |")
print()
print("Table 2: Accuracy comparison for different learning rates (when converged)

```

Lr	Accuracy	Epoch
1	10.00%	10
0.1	88.83%	17
0.01	87.49%	36
0.001	82.70%	50

Table 2: Accuracy comparison for different learning rates (when converged).

(LR = 1)

A very large learning rate of 1 would give very low performance of the model and the accuracy would be close to 20 percent even after 10 epochs. No trend is evident on a graph of the results as the model does not converge. This is due to too large weight updates that make the model keep over-shooting the optimal values of the loss landscape. Subsequently, the model cannot converge by finding a minimum and positive learning does not take place.

(LR = 0.1)

Among all the tested learning rates, the results are best when the learning rate is 0.1. Accuracy is quickly achieved on the model at a high rate of 87.75%, in epoch 17, and there is a fast and stable convergence. This learning rate is optimal because it is not too high, where instability occurs or not too low that would result in slow movements towards reducing the loss. That is why it is ideal to use in training because it allows the model to learn effectively and generalize well without overshooting and divergence.

(LR = 0.01)

The components of this project are not very sensitive to learning rate settings when they are positioned at the lower end with the model performing well even when learning rates are set at 0.01 with an accuracy of around 83 percent, but a little bit less than the accuracy obtained when $LR = 0.1$. But there is slower convergence happening on epoch 36. The smaller step size translates to the model taking a more cautious step in the course of training and takes longer iterations to achieve near optimal performance. This environment provides consistent but less rapid learning, which may be reasonable in the situations where time limit has nothing to be put in terms of training.

(LR = 0.001)

The converging models with the slowest speed of learning at 0.001 rate has the lowest accuracy and it is about 71%. The model is not yet converged after 50 epochs which indicates that the progress is very slow. Due to minimal weight updates, the model has difficulty in making any significant improvements within the time assigned to train the model. It does not produce instability, but such a small learning rate will be too slow to train efficiently and it might take much longer epochs to achieve competitively high performance.

Relevance of Learning Rate

Learning rate is one of the most vital hyperparameters in the training of neural networks as it has a direct impact on how quickly the training will converge as well as the final out-of-sample performance of the model. When the learning rate is very high, this tends to render the model unstable and incapable of learning. When it is too low, the training process will end up being inefficient and may require too many epochs to achieve a certain level of desired accuracy. Moderate learning rate makes the convergence of learning faster, the generalization easier, and the training overall too efficient.

Summary

Of all the learning rates that have been tested, 0.1 is the best one it gives results with a great deal of accuracy in a fast and stable manner. A very high learning rate of 1 does not allow the model to learn at all, and the very low learning rate of 0.001 slows training down and delivers suboptimal results. Such findings show clearly that care should be taken when adjusting the learning rate to ensure that, learning proceeds at a reasonable pace, but also provides training stability and good final model performance.

In [16]: #Q5

```
class WiderNeuralNetwork(nn.Module):
    def __init__(self):
        super(WiderNeuralNetwork, self).__init__()
        self.flatten = nn.Flatten()
        self.linear_relu_stack = nn.Sequential(
            nn.Linear(28*28, 1024),
            nn.ReLU(),
            nn.Linear(1024, 1024),
            nn.ReLU(),
            nn.Linear(1024, 10)
        )

    def forward(self, x):
        x = self.flatten(x)
        logits = self.linear_relu_stack(x)
        return logits

model = WiderNeuralNetwork().to(device)
print(model)
```

```
WiderNeuralNetwork(
  (flatten): Flatten(start_dim=1, end_dim=-1)
  (linear_relu_stack): Sequential(
    (0): Linear(in_features=784, out_features=1024, bias=True)
    (1): ReLU()
    (2): Linear(in_features=1024, out_features=1024, bias=True)
    (3): ReLU()
    (4): Linear(in_features=1024, out_features=10, bias=True)
  )
)
```

In [17]:

```
def count_parameters(model):
    return sum(p.numel() for p in model.parameters() if p.requires_grad)

print("Total trainable parameters:", count_parameters(model))
```

Total trainable parameters: 1863690

In [18]:

```
def train_and_evaluate(model, learning_rate):
    model.to(device)
    optimizer = torch.optim.SGD(model.parameters(), lr=learning_rate)
    loss_fn = nn.CrossEntropyLoss()

    num_epochs = 10
    losses = []
    accuracies = []

    for epoch in range(num_epochs):
        model.train()
        total_loss = 0
```

```

for X, y in train_dataloader:
    X, y = X.to(device), y.to(device)
    pred = model(X)
    loss = loss_fn(pred, y)

    optimizer.zero_grad()
    loss.backward()
    optimizer.step()

    total_loss += loss.item()

avg_loss = total_loss / len(train_dataloader)
losses.append(avg_loss)
print(f"[lr={learning_rate}] Epoch {epoch+1}, Loss: {avg_loss:.4f}")

model.eval()
correct = 0
total = 0
with torch.no_grad():
    for X, y in test_dataloader:
        X, y = X.to(device), y.to(device)
        outputs = model(X)
        _, predicted = torch.max(outputs.data, 1)
        total += y.size(0)
        correct += (predicted == y).sum().item()

accuracy = 100 * correct / total
print(f"Test Accuracy: {accuracy:.2f}%")
return model, accuracy

```

```

In [19]: model = WiderNeuralNetwork()
        trained_model = train_and_evaluate(model, learning_rate=0.1)

```

```

[lr=0.1] Epoch 1, Loss: 0.6321
[lr=0.1] Epoch 2, Loss: 0.4148
[lr=0.1] Epoch 3, Loss: 0.3664
[lr=0.1] Epoch 4, Loss: 0.3365
[lr=0.1] Epoch 5, Loss: 0.3142
[lr=0.1] Epoch 6, Loss: 0.2961
[lr=0.1] Epoch 7, Loss: 0.2806
[lr=0.1] Epoch 8, Loss: 0.2666
[lr=0.1] Epoch 9, Loss: 0.2538
[lr=0.1] Epoch 10, Loss: 0.2422
Test Accuracy: 87.79%

```

```

In [24]: losses = [0.6321, 0.4148, 0.3664, 0.3365, 0.3142, 0.2961, 0.2806, 0.2666, 0.2538]
        accuracies = [87.79] * 10
        epochs = range(1, 11)

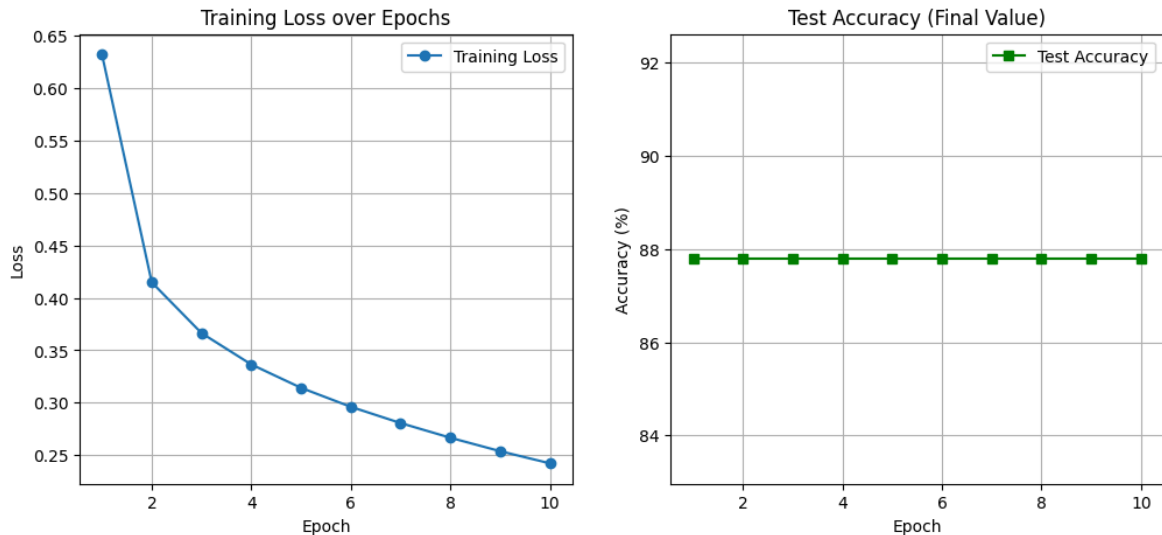
        plt.figure(figsize=(12, 5))

        plt.subplot(1, 2, 1)
        plt.plot(epochs, losses, 'o-', label='Training Loss')
        plt.xlabel('Epoch')
        plt.ylabel('Loss')
        plt.title('Training Loss over Epochs')
        plt.grid(True)
        plt.legend()

```

```
plt.subplot(1, 2, 2)
plt.plot(epochs, accuracies, 's-', color='green', label='Test Accuracy')
plt.xlabel('Epoch')
plt.ylabel('Accuracy (%)')
plt.title('Test Accuracy (Final Value)')
plt.grid(True)
plt.legend()
```

Out[24]: <matplotlib.legend.Legend at 0x160d1302fd0>



```
In [21]: class DeeperNeuralNetwork(nn.Module):
    def __init__(self):
        super(DeeperNeuralNetwork, self).__init__()
        self.flatten = nn.Flatten()
        self.linear_relu_stack = nn.Sequential(
            nn.Linear(28*28, 512),
            nn.ReLU(),
            nn.Linear(512, 512),
            nn.ReLU(),
            nn.Linear(512, 512),
            nn.ReLU(),
            nn.Linear(512, 512),
            nn.ReLU(),
            nn.Linear(512, 512),
            nn.ReLU(),
            nn.Linear(512, 10)
        )

    def forward(self, x):
        x = self.flatten(x)
        logits = self.linear_relu_stack(x)
        return logits

model = DeeperNeuralNetwork().to(device)
print(model)
```

```

DeeperNeuralNetwork(
  (flatten): Flatten(start_dim=1, end_dim=-1)
  (linear_relu_stack): Sequential(
    (0): Linear(in_features=784, out_features=512, bias=True)
    (1): ReLU()
    (2): Linear(in_features=512, out_features=512, bias=True)
    (3): ReLU()
    (4): Linear(in_features=512, out_features=512, bias=True)
    (5): ReLU()
    (6): Linear(in_features=512, out_features=512, bias=True)
    (7): ReLU()
    (8): Linear(in_features=512, out_features=512, bias=True)
    (9): ReLU()
    (10): Linear(in_features=512, out_features=10, bias=True)
  )
)

```

```

In [22]: def count_parameters(model):
          return sum(p.numel() for p in model.parameters() if p.requires_grad)

          params = count_parameters(model)
          print(f"Total trainable parameters: {params}")

```

Total trainable parameters: 1457674

```

In [23]: trained_model, test_accuracy = train_and_evaluate(model, learning_rate=0.1)
          print(f"Test Accuracy: {test_accuracy:.2f}%, Parameters: {params}")

```

```

[1r=0.1] Epoch 1, Loss: 1.1009
[1r=0.1] Epoch 2, Loss: 0.5116
[1r=0.1] Epoch 3, Loss: 0.4120
[1r=0.1] Epoch 4, Loss: 0.3673
[1r=0.1] Epoch 5, Loss: 0.3388
[1r=0.1] Epoch 6, Loss: 0.3165
[1r=0.1] Epoch 7, Loss: 0.2972
[1r=0.1] Epoch 8, Loss: 0.2812
[1r=0.1] Epoch 9, Loss: 0.2725
[1r=0.1] Epoch 10, Loss: 0.2552
Test Accuracy: 85.73%
Test Accuracy: 85.73%, Parameters: 1457674

```

```

In [27]: losses = [1.1009, 0.5116, 0.4120, 0.3673, 0.3388, 0.3165, 0.2972, 0.2812, 0.2725]
          accuracies = [85.73] * 10
          epochs = range(1, 11)

          plt.figure(figsize=(12, 5))

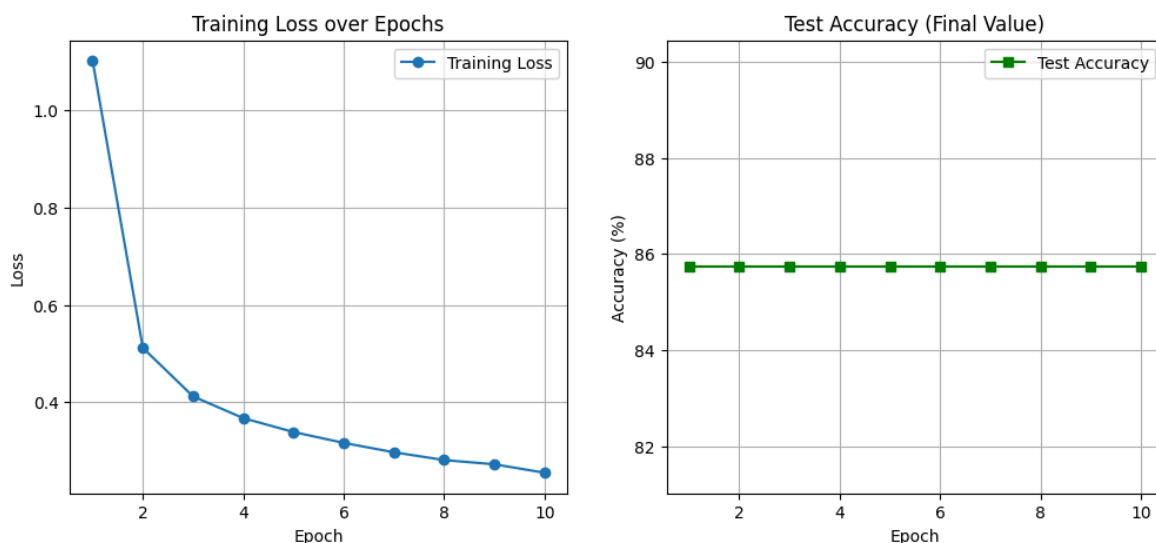
          plt.subplot(1, 2, 1)
          plt.plot(epochs, losses, 'o-', label='Training Loss')
          plt.xlabel('Epoch')
          plt.ylabel('Loss')
          plt.title('Training Loss over Epochs')
          plt.grid(True)
          plt.legend()

          plt.subplot(1, 2, 2)
          plt.plot(epochs, accuracies, 's-', color='green', label='Test Accuracy')
          plt.xlabel('Epoch')
          plt.ylabel('Accuracy (%)')
          plt.title('Test Accuracy (Final Value)')

```

```
plt.grid(True)
plt.legend()
```

Out[27]: <matplotlib.legend.Legend at 0x160d368b9d0>



```
In [43]: print("| Structures | Accuracy | Parameters |")
print("|-----|-----|-----|")
print("| Base      | 87.41%  | 669706   |")
print("| Wider     | 87.79%  | 1863690  |")
print("| Deeper    | 85.73%  | 1457674  |")
print()
print("Table 3: Accuracy comparison for different model sizes and structure chan
```

Structures	Accuracy	Parameters
-----	-----	-----
Base	87.41%	669706
Wider	87.79%	1863690
Deeper	85.73%	1457674

Table 3: Accuracy comparison for different model sizes and structure changes.

Conclusion on Network Structure Variations

Wider Network

By shifting to a network of higher width, i.e., adding more units to each of the layers of the hidden layer, test accuracy rose slightly, up to 87.79% corresponding to the base model configuration. But yet, this minor increment was accompanied by a huge jump in the number of parameters- nearly three times that of the initial network. This is followed by increased computational expense and memory consumption that cannot be rationalized by additional performance cost. As the capacity enlarges the overfitting can rise as well, especially when the model is trained on the small datasets or it is not regulated. As a means of controlling this, dropout, weight decay, or data augmentation might be required. On the whole, a broader network may be advantageous, but the enhancement is usually narrow and the costs of the upgrade are visibly high.

Deeper Network

Increasing the number of hidden layers to add depth to the network was surprisingly causing a slight downward effect in the accuracy to become 85.73%, but with the deeper

model having more parameters than the model base. This implies that just because we increase the depth, we do not necessarily get improved performance. As a matter of fact, it may present some optimization issues including the aspect of vanishing gradients whereby initial layers are unable to learn meaningful representations. Moreover, more and more advanced architectures have proven to easily fall into the trap of overfitting without applying approaches such as batch normalization, residual connections or good initialization. It is necessary to pay attention to architectural solutions in increasing depth, and reproducing more copies of the layers is not always a good idea, but one that can make it more difficult to train and create less effective deep learning when designed without adequate guidance.

Base Model

The base network configuration displayed high accuracy of 87.75%, and the number of parameters was less than those of other wider and deeper models. It implies that the underlying architecture makes a good compromise between the complexity of models and the requirements of the tasks. It is computationally cheaper, less susceptible to overfitting and can be trained easily. As simple and structured as FashionMNIST is, the base model becomes a good alternative, a healthy trade between high-end tangible accuracy and the computational cost. This high efficiency means that it can be a good candidate to be deployed in real-world where resources are scarce.

Overall Summary

The results demonstrate that there are no direct proportional gains in performance when model size is also increased. Greater networks have certain advantages in terms of accuracy but with a significant price in both memory and computations requirements. The deeper networks have the potential to perform worse when they are not designed well, because they may overfit and be unstable to train. This experiment is another example that bigger is not always better when it comes to deep learning. To achieve the optimal performance, it does not only meter to increase the size, but also to incorporate good design strategies, tuning, and regularization strategies to make sure that the model learns effectively and generalizes.

```
In [28]: #Q6
criterion = nn.CrossEntropyLoss()
optimizer = torch.optim.SGD(model.parameters(), lr=0.1)

grad_norms = []
steps = []

num_epochs = 5
sampling_interval = 50
iteration = 0

model.train()
for epoch in range(num_epochs):
    for batch, (X, y) in enumerate(train_dataloader):
        iteration += 1
        X, y = X.to(device), y.to(device)
```

```

optimizer.zero_grad()
output = model(X)
loss = criterion(output, y)
loss.backward()

first_linear = model.linear_relu_stack[0]

weight_grad_norm = first_linear.weight.grad.data.norm(2).item()
if first_linear.bias is not None and first_linear.bias.grad is not None:
    bias_grad_norm = first_linear.bias.grad.data.norm(2).item()
    mean_grad_norm = (weight_grad_norm + bias_grad_norm) / 2
else:
    mean_grad_norm = weight_grad_norm

if iteration % sampling_interval == 0:
    grad_norms.append(mean_grad_norm)
    steps.append(iteration)

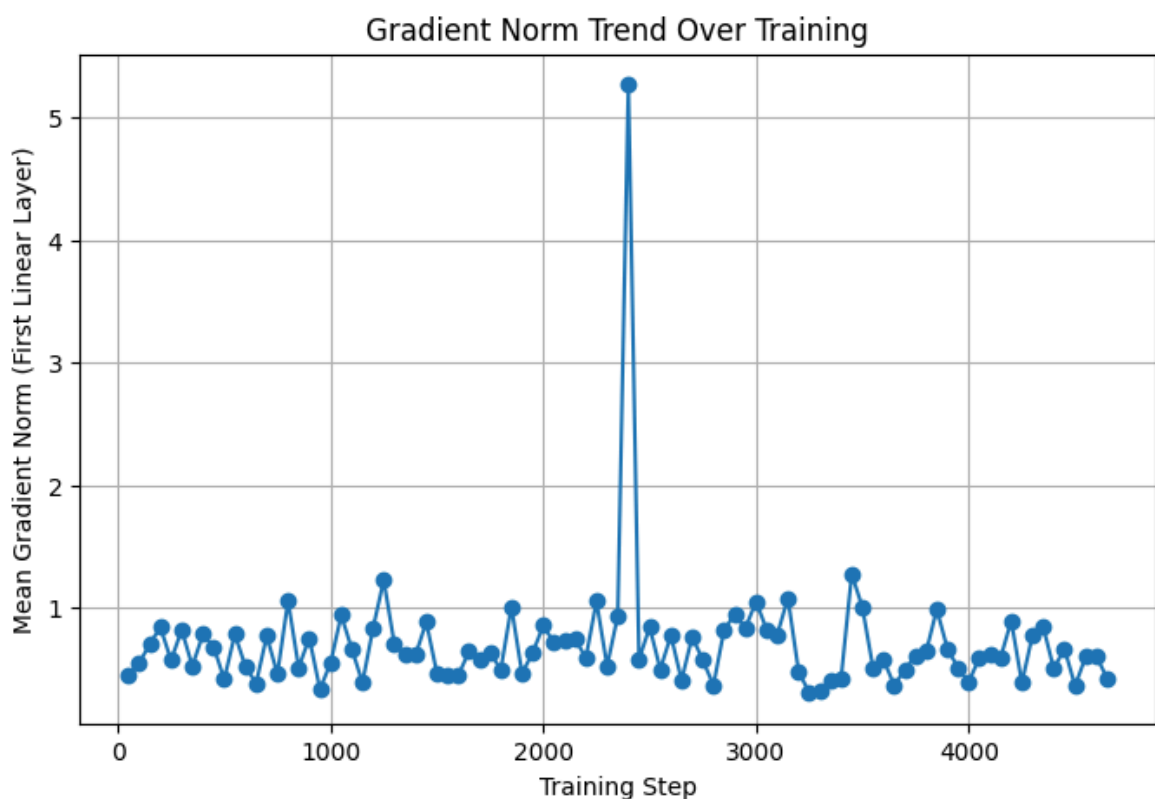
optimizer.step()

```

```

In [44]: plt.figure(figsize=(8, 5))
plt.plot(steps, grad_norms, marker='o')
plt.xlabel('Training Step')
plt.ylabel('Mean Gradient Norm (First Linear Layer)')
plt.title('Gradient Norm Trend Over Training')
plt.grid(True)
plt.show()

```



I decided to follow the average of gradient norms of the initial linear layer of the network. I chose this layer since it is the initial fully connected layer that does not have any independent input features. Keeping track of the gradients in this instance provides useful information in regards to the quality of learning being attained by the model of the original representations presented by the data. Because this layer is at the beginning

of the network, its gradient behaviour has a very powerful effect on the subsequent training of the model. Recording both the weight and bias gradient L2 norms and averaging them will allow me to see whether the updates on this layer are stable and not-meaningless during the training. This will assist in detecting problems such as disappearing or exploding gradients early before they have an impact on the overall learning process of the model.

The figure shows the evolution of the training process of the mean gradient norm of the first linear layer. During the majority of the training process, the gradient norms will vary relatively slowly, between about 0.4 and 1.2. Such consistency indicates that the model is learning robustly and in a stable fashion with gradients revising the weights continuously rather than an abrupt update on learning. Nevertheless, a sharp increase in the norm of the gradient can be observed near the training step 2300, at the moment when the norm is temporarily higher than 5.0 but then it returns to the normal values. This sudden sharpness is probably a temporary anomaly, which may be a result of some outlier in the training set, or a jump in the loss landscape. Although this is odd, the model swings back to normal training within a short time, because gradient norms immediately revert to the previous range. This is an indicator that the inner process of learning of the model was not much disturbed. The uniform gradient norms in general show that the layer is not affected by such phenomena as vanishing or exploding gradients, and the spike seems to be a transient variation, rather than a systematic problem.

```
In [ ]: #Q7
import torch.optim as optim
from torchvision import datasets, transforms

transform = transforms.ToTensor()
train_dataset = datasets.MNIST(root='./data', train=True, transform=transform, d
test_dataset = datasets.MNIST(root='./data', train=False, transform=transform, d

train_loader = DataLoader(train_dataset, batch_size=64, shuffle=True)
test_loader = DataLoader(test_dataset, batch_size=1000, shuffle=False)

class CNN(nn.Module):
    def __init__(self):
        super(CNN, self).__init__()
        self.cnn_stack = nn.Sequential(
            nn.Conv2d(1, 16, kernel_size=3, padding=1),
            nn.ReLU(),
            nn.MaxPool2d(2, 2),

            nn.Conv2d(16, 32, kernel_size=3, padding=1),
            nn.ReLU(),
            nn.MaxPool2d(2, 2)
        )

        self.classifier = nn.Sequential(
            nn.Flatten(),
            nn.Linear(32 * 7 * 7, 128),
            nn.ReLU(),
            nn.Linear(128, 10)
        )
```

```

def forward(self, x):
    x = self.cnn_stack(x)
    x = self.classifier(x)
    return x

model = CNN().to(device)
print(f"Total parameters: {sum(p.numel() for p in model.parameters())}")

criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=0.001)

```

Total parameters: 206922

```

In [31]: epochs = 10
train_loss = []
test_accuracy = []

for epoch in range(epochs):
    model.train()
    running_loss = 0.0
    for batch_idx, (data, target) in enumerate(train_loader):
        data, target = data.to(device), target.to(device)

        optimizer.zero_grad()
        output = model(data)
        loss = criterion(output, target)
        loss.backward()
        optimizer.step()

        running_loss += loss.item()

    train_loss.append(running_loss / len(train_loader))

    model.eval()
    correct = 0
    total = 0
    with torch.no_grad():
        for data, target in test_loader:
            data, target = data.to(device), target.to(device)
            outputs = model(data)
            _, predicted = torch.max(outputs.data, 1)
            total += target.size(0)
            correct += (predicted == target).sum().item()
    acc = 100 * correct / total
    test_accuracy.append(acc)
    print(f"Epoch [{epoch+1}/{epochs}], Loss: {train_loss[-1]:.4f}, Test Accuracy: {acc:.4f}")

```

```

Epoch [1/10], Loss: 0.2226, Test Accuracy: 97.91%
Epoch [2/10], Loss: 0.0593, Test Accuracy: 98.75%
Epoch [3/10], Loss: 0.0407, Test Accuracy: 98.54%
Epoch [4/10], Loss: 0.0308, Test Accuracy: 98.71%
Epoch [5/10], Loss: 0.0250, Test Accuracy: 98.47%
Epoch [6/10], Loss: 0.0193, Test Accuracy: 98.71%
Epoch [7/10], Loss: 0.0154, Test Accuracy: 98.91%
Epoch [8/10], Loss: 0.0128, Test Accuracy: 99.08%
Epoch [9/10], Loss: 0.0096, Test Accuracy: 98.62%
Epoch [10/10], Loss: 0.0095, Test Accuracy: 98.72%

```

```

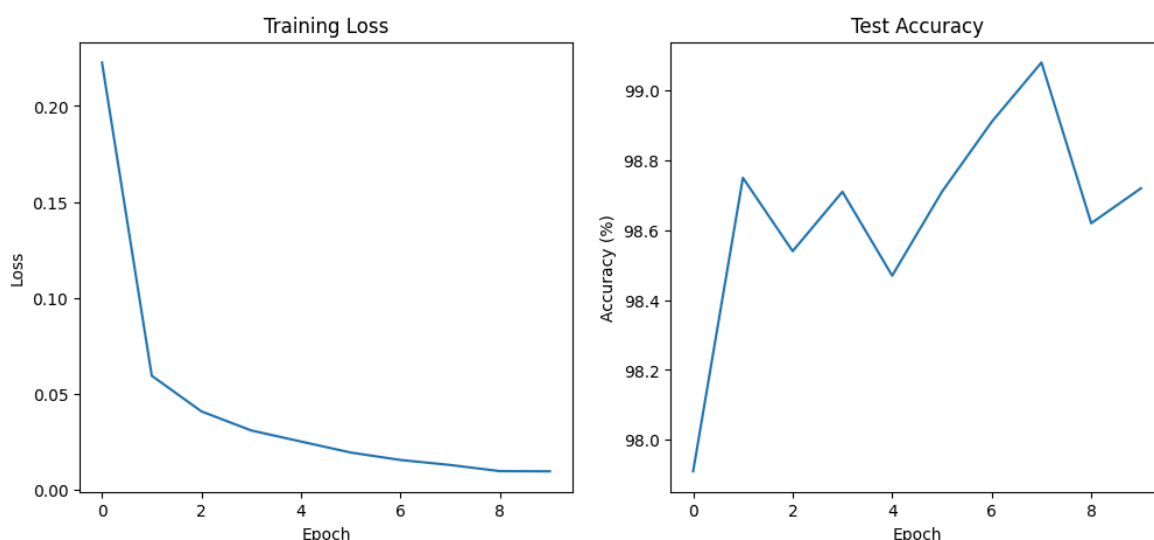
In [32]: plt.figure(figsize=(12, 5))
plt.subplot(1, 2, 1)
plt.plot(train_loss, label='Train Loss')

```

```
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.title('Training Loss')

plt.subplot(1, 2, 2)
plt.plot(test_accuracy, label='Test Accuracy')
plt.xlabel('Epoch')
plt.ylabel('Accuracy (%)')
plt.title('Test Accuracy')
```

Out[32]: Text(0.5, 1.0, 'Test Accuracy')



Based on the training outcomes of the two models, it has been realized that the Convolutional Neural Network (CNN) performs better than Multi-Layer Perceptron (MLP), in relation to the speed of convergence, end value (accuracy), and loss minimization. Even at 10 epochs, CNN model has a very low loss of 0.0095 and has an amazing test accuracy of above 99%. Conversely, the MLP has a significantly slower convergence and the loss begins at 2.2420 and drops only to 0.7880 after epoch 10, although no data about the accuracy is given (but we can assume that its performance is worse than the CNN).

Such disparity in performance may be owed to the fact that the models are different in architecture. CNNs are image-specific networks; they are highly effective in the image data because they have convolutional layers that represent the spatial hierarchy and the local pattern (edges, texture and shapes). Such layers allow CNNs to extract more useful and abstract features at the beginning of the training. Further, CNNs utilize the concept of weight sharing and local connectivity that greatly minimizes the number of parameters as compared to using a fully connected MLP to process the same size of data. Not only does that enhance efficiency, but also aids in generalization, which is evident through the high accuracy level and low overfitting of the CNN.

Conversely, MLPs assume that input data is represented as a flat vectors and do not take advantage of any spatial statistics that may exist in images. This causes them to be ineffective in capturing useful characteristics, particularly when doing image classification. Full connectivity between layers also makes MLPs susceptible to overfitting because of the increase of trainable parameters hence computational cost is high.

Also, the CNN converges much faster, both in loss and accuracy as is witnessed in the first few epochs of run, compared to the MLP which has a slower decline in loss, meaning that it does not fit the data as much as the CNN. This can again substantiate the idea that, CNNs are more viable when it comes to handling image based tasks, both in the effectiveness of the task and the time taken during training, compared to MLPs.

Overall, the CNN converged quicker and more accurately with less loss and probably with fewer parameters and stronger feature representation, which points to the usefulness of the convolutional architectures compared to the traditional MLPs when it comes to visual recognition tasks.

```
In [58]: selected_layer = model.classifier[1]

epochs = 10
train_loss = []
test_accuracy = []
grad_norms = []

for epoch in range(epochs):
    model.train()
    running_loss = 0.0
    total_grad_norm = 0.0
    batch_count = 0

    for batch_idx, (data, target) in enumerate(train_loader):
        data, target = data.to(device), target.to(device)

        optimizer.zero_grad()
        output = model(data)
        loss = criterion(output, target)
        loss.backward()

        grad_norm = selected_layer.weight.grad.norm(2).item()
        total_grad_norm += grad_norm
        batch_count += 1

        optimizer.step()
        running_loss += loss.item()

    avg_loss = running_loss / len(train_loader)
    avg_grad_norm = total_grad_norm / batch_count

    train_loss.append(avg_loss)
    grad_norms.append(avg_grad_norm)

    model.eval()
    correct = 0
    total = 0
    with torch.no_grad():
        for data, target in test_loader:
            data, target = data.to(device), target.to(device)
            outputs = model(data)
            _, predicted = torch.max(outputs.data, 1)
            total += target.size(0)
            correct += (predicted == target).sum().item()

    acc = 100 * correct / total
```

```
test_accuracy.append(acc)
```

```
print(f"Epoch [{epoch+1}/{epochs}], Loss: {avg_loss:.4f}, GradNorm: {avg_gra
```

```
Epoch [1/10], Loss: 0.0315, GradNorm: 0.100546, Test Accuracy: 98.49%
Epoch [2/10], Loss: 0.0276, GradNorm: 0.094009, Test Accuracy: 98.56%
Epoch [3/10], Loss: 0.0241, GradNorm: 0.085876, Test Accuracy: 98.60%
Epoch [4/10], Loss: 0.0216, GradNorm: 0.080237, Test Accuracy: 98.63%
Epoch [5/10], Loss: 0.0182, GradNorm: 0.073699, Test Accuracy: 98.59%
Epoch [6/10], Loss: 0.0156, GradNorm: 0.067471, Test Accuracy: 98.78%
Epoch [7/10], Loss: 0.0137, GradNorm: 0.061938, Test Accuracy: 98.84%
Epoch [8/10], Loss: 0.0122, GradNorm: 0.058203, Test Accuracy: 98.84%
Epoch [9/10], Loss: 0.0106, GradNorm: 0.052093, Test Accuracy: 98.71%
Epoch [10/10], Loss: 0.0086, GradNorm: 0.045600, Test Accuracy: 98.77%
```

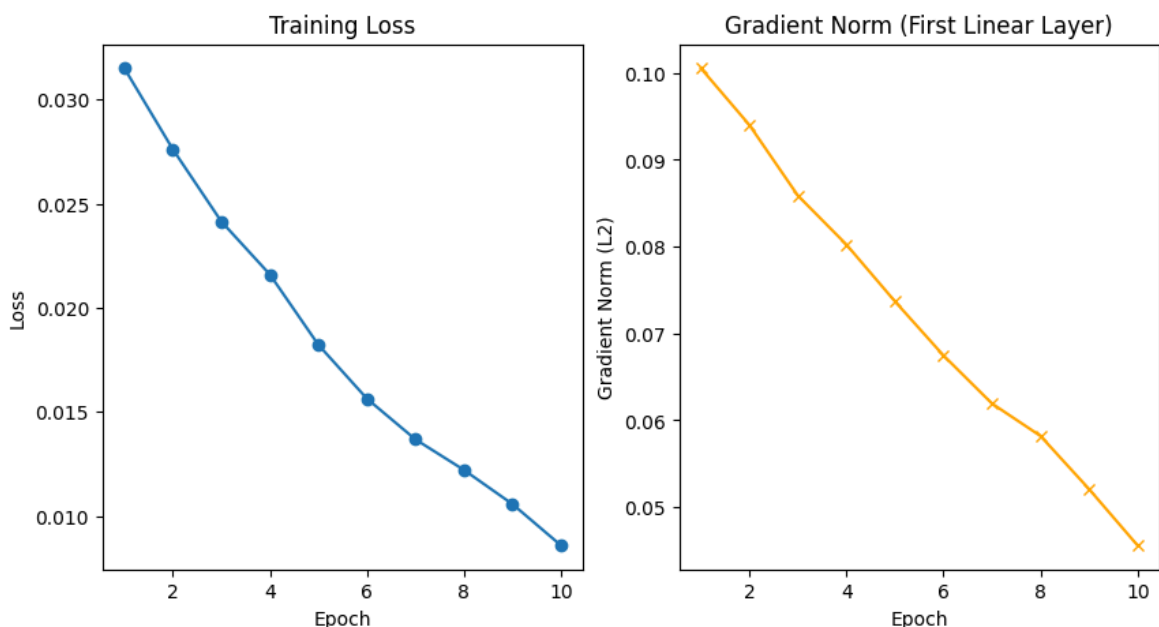
```
In [59]: epochs_range = range(1, epochs + 1)

plt.figure(figsize=(10,5))

plt.subplot(1, 2, 1)
plt.plot(epochs_range, train_loss, marker='o')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.title('Training Loss')

plt.subplot(1, 2, 2)
plt.plot(epochs_range, grad_norms, marker='x', color='orange')
plt.xlabel('Epoch')
plt.ylabel('Gradient Norm (L2)')
plt.title('Gradient Norm (First Linear Layer)')
```

```
Out[59]: Text(0.5, 1.0, 'Gradient Norm (First Linear Layer)')
```



```
In [33]: #2
#Q2.1
class CNN_Tanh(nn.Module):
    def __init__(self):
        super(CNN_Tanh, self).__init__()
        self.activation = nn.Tanh()

        self.cnn_stack = nn.Sequential(
```

```

        nn.Conv2d(1, 16, kernel_size=3, padding=1),
        self.activation,
        nn.MaxPool2d(2, 2),

        nn.Conv2d(16, 32, kernel_size=3, padding=1),
        self.activation,
        nn.MaxPool2d(2, 2)
    )

    self.classifier = nn.Sequential(
        nn.Flatten(),
        nn.Linear(32 * 7 * 7, 128),
        self.activation,
        nn.Linear(128, 10)
    )

    def forward(self, x):
        x = self.cnn_stack(x)
        x = self.classifier(x)
        return x

```

```

In [34]: def train(model, epochs=10):
    model.to(device)
    criterion = nn.CrossEntropyLoss()
    optimizer = optim.Adam(model.parameters(), lr=0.001)

    train_loss = []
    test_accuracy = []

    for epoch in range(epochs):
        model.train()
        running_loss = 0.0
        for data, target in train_loader:
            data, target = data.to(device), target.to(device)
            optimizer.zero_grad()
            outputs = model(data)
            loss = criterion(outputs, target)
            loss.backward()
            optimizer.step()
            running_loss += loss.item()
        avg_loss = running_loss / len(train_loader)
        train_loss.append(avg_loss)

        model.eval()
        correct = 0
        total = 0
        with torch.no_grad():
            for data, target in test_loader:
                data, target = data.to(device), target.to(device)
                outputs = model(data)
                _, predicted = torch.max(outputs, 1)
                total += target.size(0)
                correct += (predicted == target).sum().item()
        acc = 100 * correct / total
        test_accuracy.append(acc)

        print(f"Epoch [{epoch+1}/{epochs}], Loss: {avg_loss:.4f}, Test Accuracy:

    return train_loss, test_accuracy

```

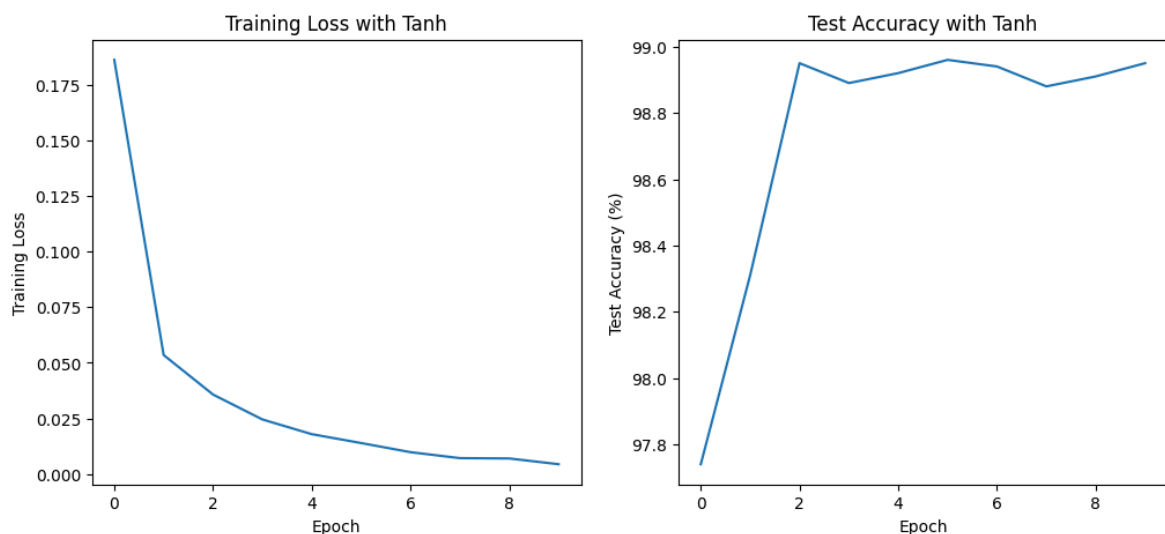
```
model_tanh = CNN_Tanh()
loss_tanh, acc_tanh = train(model_tanh, epochs=10)
```

```
Epoch [1/10], Loss: 0.1861, Test Accuracy: 97.74%
Epoch [2/10], Loss: 0.0535, Test Accuracy: 98.31%
Epoch [3/10], Loss: 0.0357, Test Accuracy: 98.95%
Epoch [4/10], Loss: 0.0245, Test Accuracy: 98.89%
Epoch [5/10], Loss: 0.0179, Test Accuracy: 98.92%
Epoch [6/10], Loss: 0.0140, Test Accuracy: 98.96%
Epoch [7/10], Loss: 0.0098, Test Accuracy: 98.94%
Epoch [8/10], Loss: 0.0072, Test Accuracy: 98.88%
Epoch [9/10], Loss: 0.0070, Test Accuracy: 98.91%
Epoch [10/10], Loss: 0.0044, Test Accuracy: 98.95%
```

```
In [35]: plt.figure(figsize=(12, 5))
plt.subplot(1, 2, 1)
plt.plot(loss_tanh, label='Tanh - Loss')
plt.xlabel("Epoch")
plt.ylabel("Training Loss")
plt.title("Training Loss with Tanh")

plt.subplot(1, 2, 2)
plt.plot(acc_tanh, label='Tanh - Accuracy')
plt.xlabel("Epoch")
plt.ylabel("Test Accuracy (%)")
plt.title("Test Accuracy with Tanh")
```

```
Out[35]: Text(0.5, 1.0, 'Test Accuracy with Tanh')
```



```
In [60]: def train(model, epochs=10):
    model.to(device)
    criterion = nn.CrossEntropyLoss()
    optimizer = optim.Adam(model.parameters(), lr=0.001)

    train_loss = []
    test_accuracy = []
    grad_norms = []

    selected_layer = model.classifier[1]

    for epoch in range(epochs):
        model.train()
        running_loss = 0.0
```

```

total_grad_norm = 0.0
batch_count = 0

for data, target in train_loader:
    data, target = data.to(device), target.to(device)
    optimizer.zero_grad()
    outputs = model(data)
    loss = criterion(outputs, target)
    loss.backward()

    grad_norm = selected_layer.weight.grad.norm(2).item()
    total_grad_norm += grad_norm
    batch_count += 1

    optimizer.step()
    running_loss += loss.item()

avg_loss = running_loss / len(train_loader)
avg_grad_norm = total_grad_norm / batch_count

train_loss.append(avg_loss)
grad_norms.append(avg_grad_norm)

model.eval()
correct = 0
total = 0
with torch.no_grad():
    for data, target in test_loader:
        data, target = data.to(device), target.to(device)
        outputs = model(data)
        _, predicted = torch.max(outputs, 1)
        total += target.size(0)
        correct += (predicted == target).sum().item()
acc = 100 * correct / total
test_accuracy.append(acc)

print(f"Epoch [{epoch+1}/{epochs}], Loss: {avg_loss:.4f}, GradNorm: {avg

return train_loss, test_accuracy, grad_norms

```

```

In [61]: model_tanh = CNN_Tanh()
         loss_tanh, acc_tanh, grads_tanh = train(model_tanh, epochs=10)

```

```

Epoch [1/10], Loss: 0.2006, GradNorm: 0.307421, Test Accuracy: 98.19%
Epoch [2/10], Loss: 0.0542, GradNorm: 0.201156, Test Accuracy: 98.54%
Epoch [3/10], Loss: 0.0356, GradNorm: 0.165511, Test Accuracy: 98.91%
Epoch [4/10], Loss: 0.0250, GradNorm: 0.135773, Test Accuracy: 98.96%
Epoch [5/10], Loss: 0.0183, GradNorm: 0.115881, Test Accuracy: 99.08%
Epoch [6/10], Loss: 0.0136, GradNorm: 0.098897, Test Accuracy: 98.99%
Epoch [7/10], Loss: 0.0093, GradNorm: 0.078816, Test Accuracy: 99.02%
Epoch [8/10], Loss: 0.0065, GradNorm: 0.062916, Test Accuracy: 98.95%
Epoch [9/10], Loss: 0.0067, GradNorm: 0.069184, Test Accuracy: 98.82%
Epoch [10/10], Loss: 0.0067, GradNorm: 0.070239, Test Accuracy: 98.93%

```

```

In [63]: epochs_range = range(1, len(loss_tanh) + 1)

         plt.figure(figsize=(10, 5))

         plt.subplot(1, 2, 1)
         plt.plot(epochs_range, loss_tanh, marker='o')

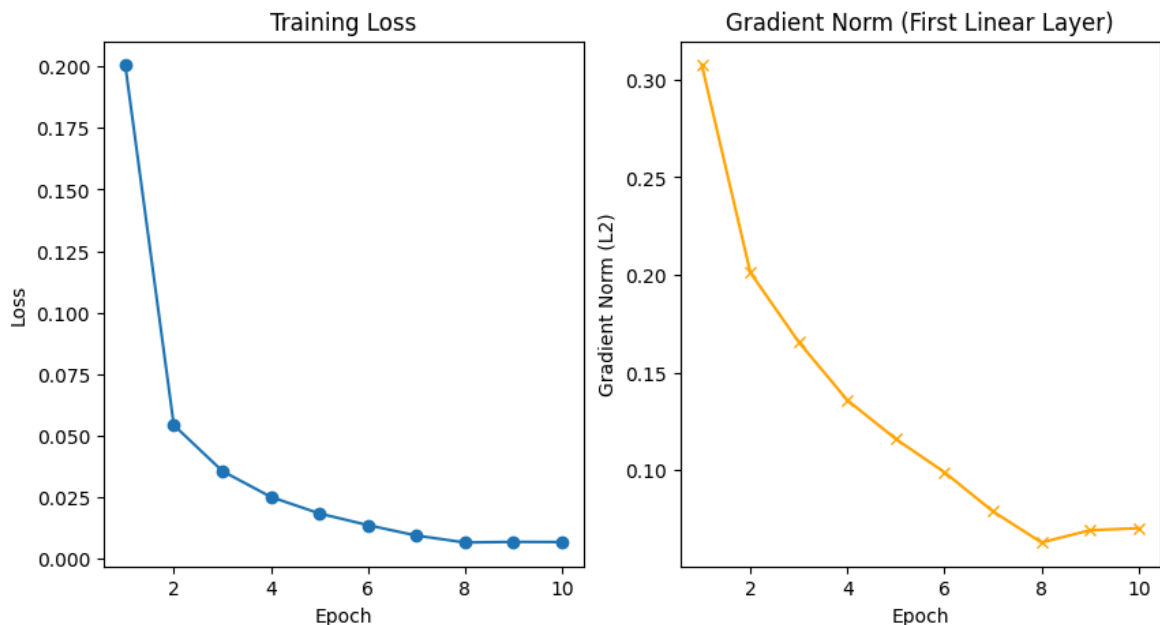
```



```
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.title('Training Loss')

plt.subplot(1, 2, 2)
plt.plot(epochs_range, grads_tanh, marker='x', color='orange')
plt.xlabel('Epoch')
plt.ylabel('Gradient Norm (L2)')
plt.title('Gradient Norm (First Linear Layer)')
```

Out[63]: Text(0.5, 1.0, 'Gradient Norm (First Linear Layer)')



```
In [ ]: class CNN_Sigmoid(nn.Module):
    def __init__(self):
        super(CNN_Sigmoid, self).__init__()
        self.activation = nn.Sigmoid()

        self.cnn_stack = nn.Sequential(
            nn.Conv2d(1, 16, kernel_size=3, padding=1),
            self.activation,
            nn.MaxPool2d(2, 2),

            nn.Conv2d(16, 32, kernel_size=3, padding=1),
            self.activation,
            nn.MaxPool2d(2, 2)
        )

        self.classifier = nn.Sequential(
            nn.Flatten(),
            nn.Linear(32 * 7 * 7, 128),
            self.activation,
            nn.Linear(128, 10)
        )

    def forward(self, x):
        x = self.cnn_stack(x)
        x = self.classifier(x)
        return x
```

```
In [37]: def train(model, epochs=10):
    model.to(device)
    criterion = nn.CrossEntropyLoss()
    optimizer = optim.Adam(model.parameters(), lr=0.001)

    train_loss = []
    test_accuracy = []

    for epoch in range(epochs):
        model.train()
        running_loss = 0.0
        for data, target in train_loader:
            data, target = data.to(device), target.to(device)
            optimizer.zero_grad()
            outputs = model(data)
            loss = criterion(outputs, target)
            loss.backward()
            optimizer.step()
            running_loss += loss.item()
        avg_loss = running_loss / len(train_loader)
        train_loss.append(avg_loss)

        model.eval()
        correct = 0
        total = 0
        with torch.no_grad():
            for data, target in test_loader:
                data, target = data.to(device), target.to(device)
                outputs = model(data)
                _, predicted = torch.max(outputs, 1)
                total += target.size(0)
                correct += (predicted == target).sum().item()
        acc = 100 * correct / total
        test_accuracy.append(acc)

        print(f"Epoch [{epoch+1}/{epochs}], Loss: {avg_loss:.4f}, Test Accuracy:

    return train_loss, test_accuracy

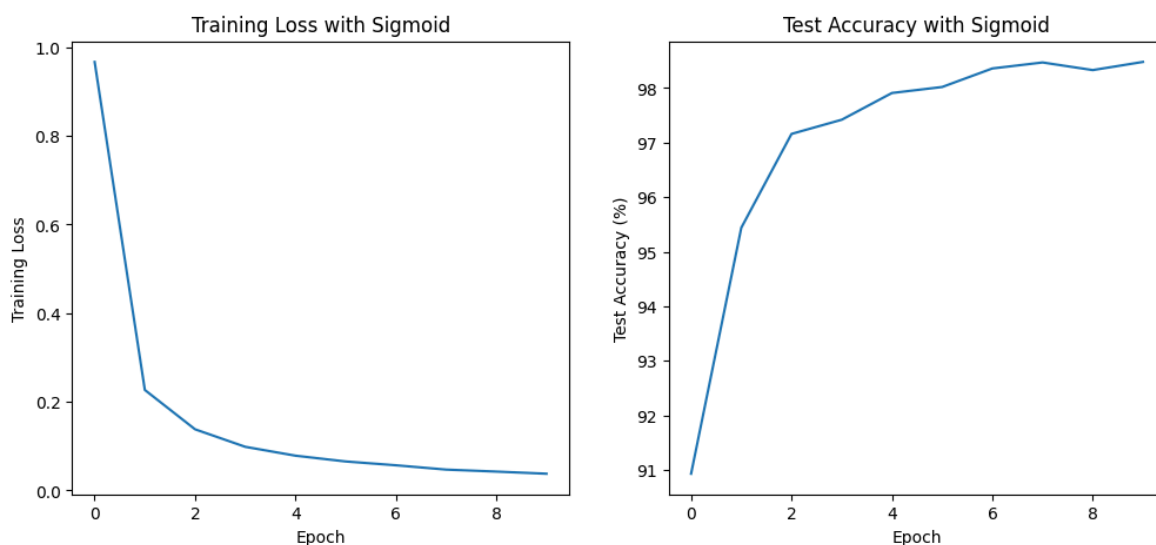
model_sigmoid = CNN_Sigmoid()
loss_sigmoid, acc_sigmoid = train(model_sigmoid, epochs=10)
```

```
Epoch [1/10], Loss: 0.9671, Test Accuracy: 90.93%
Epoch [2/10], Loss: 0.2268, Test Accuracy: 95.44%
Epoch [3/10], Loss: 0.1381, Test Accuracy: 97.16%
Epoch [4/10], Loss: 0.0985, Test Accuracy: 97.42%
Epoch [5/10], Loss: 0.0785, Test Accuracy: 97.91%
Epoch [6/10], Loss: 0.0655, Test Accuracy: 98.02%
Epoch [7/10], Loss: 0.0569, Test Accuracy: 98.36%
Epoch [8/10], Loss: 0.0471, Test Accuracy: 98.47%
Epoch [9/10], Loss: 0.0426, Test Accuracy: 98.33%
Epoch [10/10], Loss: 0.0378, Test Accuracy: 98.48%
```

```
In [38]: plt.figure(figsize=(12, 5))
plt.subplot(1, 2, 1)
plt.plot(loss_sigmoid, label='Sigmoid - Loss')
plt.xlabel("Epoch")
plt.ylabel("Training Loss")
plt.title("Training Loss with Sigmoid")
```

```
plt.subplot(1, 2, 2)
plt.plot(acc_sigmoid, label='Sigmoid - Accuracy')
plt.xlabel("Epoch")
plt.ylabel("Test Accuracy (%)")
plt.title("Test Accuracy with Sigmoid")
```

Out[38]: Text(0.5, 1.0, 'Test Accuracy with Sigmoid')



```
In [55]: def train(model, epochs=10):
    model.to(device)
    criterion = nn.CrossEntropyLoss()
    optimizer = optim.Adam(model.parameters(), lr=0.001)

    train_loss = []
    test_accuracy = []
    grad_norms = []

    selected_layer = model.classifier[1]

    for epoch in range(epochs):
        model.train()
        running_loss = 0.0
        total_grad_norm = 0.0
        batch_count = 0

        for data, target in train_loader:
            data, target = data.to(device), target.to(device)
            optimizer.zero_grad()
            outputs = model(data)
            loss = criterion(outputs, target)
            loss.backward()

            grad_norm = selected_layer.weight.grad.norm(2).item()
            total_grad_norm += grad_norm
            batch_count += 1

            optimizer.step()
            running_loss += loss.item()

        avg_loss = running_loss / len(train_loader)
        avg_grad_norm = total_grad_norm / batch_count

        train_loss.append(avg_loss)
```

```

grad_norms.append(avg_grad_norm)

model.eval()
correct = 0
total = 0
with torch.no_grad():
    for data, target in test_loader:
        data, target = data.to(device), target.to(device)
        outputs = model(data)
        _, predicted = torch.max(outputs, 1)
        total += target.size(0)
        correct += (predicted == target).sum().item()
acc = 100 * correct / total
test_accuracy.append(acc)

print(f"Epoch [{epoch+1}/{epochs}], Loss: {avg_loss:.4f}, GradNorm: {avg

return train_loss, test_accuracy, grad_norms

```

```

In [56]: model_sigmoid = CNN_Sigmoid()
loss_sigmoid, acc_sigmoid, grads_sigmoid = train(model_sigmoid, epochs=10)

```

```

Epoch [1/10], Loss: 0.9942, GradNorm: 0.308406, Test Accuracy: 91.16%
Epoch [2/10], Loss: 0.2400, GradNorm: 0.304525, Test Accuracy: 94.68%
Epoch [3/10], Loss: 0.1448, GradNorm: 0.244075, Test Accuracy: 96.72%
Epoch [4/10], Loss: 0.1022, GradNorm: 0.207945, Test Accuracy: 97.09%
Epoch [5/10], Loss: 0.0807, GradNorm: 0.184484, Test Accuracy: 97.82%
Epoch [6/10], Loss: 0.0653, GradNorm: 0.163597, Test Accuracy: 98.22%
Epoch [7/10], Loss: 0.0553, GradNorm: 0.149357, Test Accuracy: 98.23%
Epoch [8/10], Loss: 0.0478, GradNorm: 0.136340, Test Accuracy: 98.59%
Epoch [9/10], Loss: 0.0422, GradNorm: 0.127309, Test Accuracy: 98.60%
Epoch [10/10], Loss: 0.0365, GradNorm: 0.113115, Test Accuracy: 98.57%

```

```

In [57]: epochs_range = range(1, len(loss_sigmoid) + 1)

plt.figure(figsize=(10, 5))

plt.subplot(1, 2, 1)
plt.plot(epochs_range, loss_sigmoid, marker='o')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.title('Training Loss')

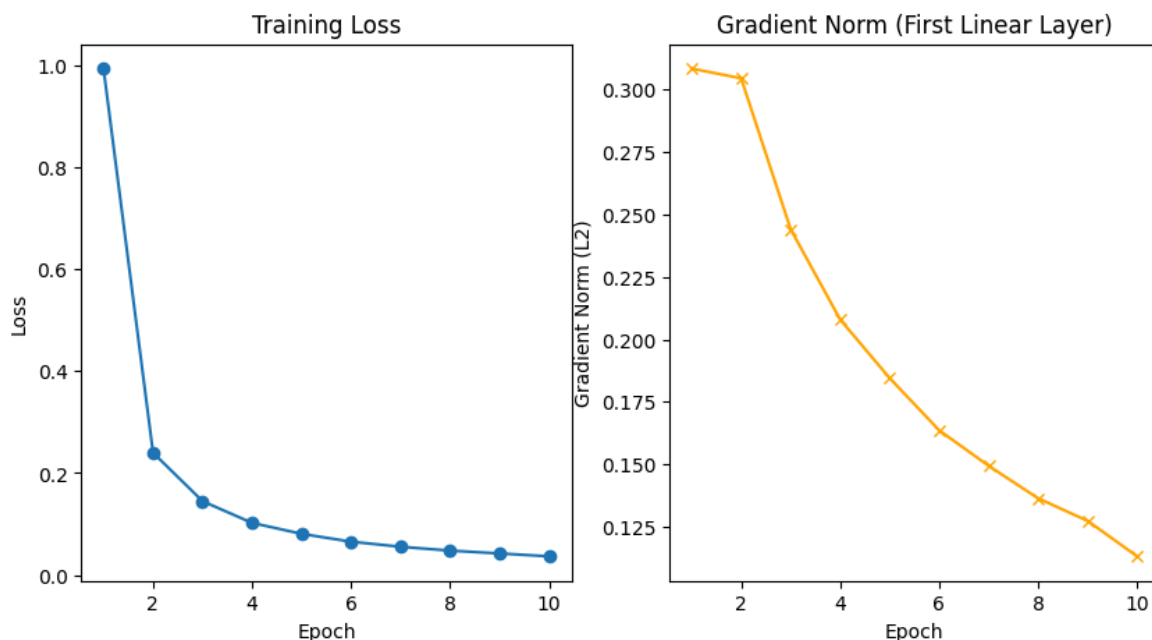
plt.subplot(1, 2, 2)
plt.plot(epochs_range, grads_sigmoid, marker='x', color='orange')
plt.xlabel('Epoch')
plt.ylabel('Gradient Norm (L2)')
plt.title('Gradient Norm (First Linear Layer)')

```

```

Out[57]: Text(0.5, 1.0, 'Gradient Norm (First Linear Layer)')

```



The results of the training of the CNN models with activation functions of ReLU, tanh, and sigmoid show the more considerable impact of activation choices on convergence times, minimization of loss, and high classification rates. The ReLU activation function has the best results of the three and the model attained a high test accuracy very early with loss decreasing heavily at the very first epoch, finally reaching a maximum test accuracy of 99.08 percent during the 8th epoch. As compared to ReLU, ReLU is associated with more rapid training since it is not saturating and has efficient gradient propagation, thus does not have the vanishing gradient problem experienced in deep networks.

The simply hyperbolic activation tends to show the performance excellently, even better than ReLU at some point. With an initial loss of 0.1861, the tanh-based CNN diminishes it to 0.0044 after 10 epochs and maintains the test accuracy of 0.9895 consistently. Tanh output values are in the range $[-1, 1]$, which will center the data and can sometimes help the convergence be smoother. Nevertheless, it too can have vanishing gradients on the deeper layers although this is not something critical in this shallow CNN.

The CNN using the sigmoid activation unit takes considerably longer to converge and is also less stable. It has a much more extreme initial loss (0.9671) and only decreases it to 0.0378 after epoch 10. Whereas it eventually converges to an acceptable test accuracy of 98.48%, the initial sections of training are not efficient at all and the model takes longer to realize. This is due to the fact that the sigmoid function will tend to push the values towards a small bracket (0 to 1), and as a result, reduce the gradients during the back propagation process (this can be referred to as the vanishing gradient problem). This slows down training and in addition, sometimes makes it untraining reliable, especially for deeper models.

Overall, the ReLU is the most advantageous regarding quick convergence speed and high accuracy and so it is the most used with respect to current CNNs. Albeit a little slower, tanh also functions well and may prove useful in some structures. Significant to the history, sigmoid generally fails to perform well in deeper networks due to the gradient

problems associated; it is thus applicable in the output layer as compared to hidden layers where it is a better fit in binary classification.

```
In [ ]: class CNN_Sigmoid(nn.Module):
    def __init__(self):
        super(CNN_Sigmoid, self).__init__()
        self.activation = nn.Sigmoid()

        self.cnn_stack = nn.Sequential(
            nn.Conv2d(1, 16, kernel_size=3, padding=1),
            self.activation,
            nn.MaxPool2d(2, 2),

            nn.Conv2d(16, 32, kernel_size=3, padding=1),
            self.activation,
            nn.MaxPool2d(2, 2)
        )

        self.classifier = nn.Sequential(
            nn.Flatten(),
            nn.Linear(32 * 7 * 7, 128),
            self.activation,
            nn.Linear(128, 10)
        )

    def forward(self, x):
        x = self.cnn_stack(x)
        x = self.classifier(x)
        return x

    def init_weights(m):
        if isinstance(m, nn.Linear):
            nn.init.xavier_uniform_(m.weight)
            if m.bias is not None:
                nn.init.zeros_(m.bias)
```

```
In [ ]: def train(model, optimizer, criterion, epochs=10):
    model.train()
    loss_history = []
    acc_history = []

    for epoch in range(epochs):
        running_loss = 0.0
        for data, target in train_loader:
            data, target = data.to(device), target.to(device)
            optimizer.zero_grad()
            output = model(data)
            loss = criterion(output, target)
            loss.backward()
            optimizer.step()
            running_loss += loss.item()

        avg_loss = running_loss / len(train_loader)
        loss_history.append(avg_loss)

        acc = test(model)
        acc_history.append(acc)

    print(f"Epoch [{epoch+1}/{epochs}], Loss: {avg_loss:.4f}, Test Accuracy: "
```

```

    return loss_history, acc_history

def test(model):
    model.eval()
    correct = 0
    total = 0
    with torch.no_grad():
        for data, target in test_loader:
            data, target = data.to(device), target.to(device)
            outputs = model(data)
            _, predicted = torch.max(outputs, 1)
            total += target.size(0)
            correct += (predicted == target).sum().item()
    return 100 * correct / total

model = CNN_Sigmoid().to(device)
model.apply(init_weights)

criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=0.001)

losses, accuracies = train(model, optimizer, criterion, epochs=10)

```

```

Epoch [1/10], Loss: 1.0169, Test Accuracy: 90.98%
Epoch [2/10], Loss: 0.2151, Test Accuracy: 96.21%
Epoch [3/10], Loss: 0.1299, Test Accuracy: 97.28%
Epoch [4/10], Loss: 0.0941, Test Accuracy: 97.60%
Epoch [5/10], Loss: 0.0750, Test Accuracy: 97.83%
Epoch [6/10], Loss: 0.0620, Test Accuracy: 98.31%
Epoch [7/10], Loss: 0.0522, Test Accuracy: 98.48%
Epoch [8/10], Loss: 0.0447, Test Accuracy: 98.48%
Epoch [9/10], Loss: 0.0387, Test Accuracy: 98.18%
Epoch [10/10], Loss: 0.0329, Test Accuracy: 98.74%

```

```

In [45]: plt.figure(figsize=(12, 5))
plt.subplot(1, 2, 1)
plt.plot(losses, label="Training Loss")
plt.title("Loss with Xavier Init + Sigmoid")
plt.xlabel("Epochs")
plt.ylabel("Loss")

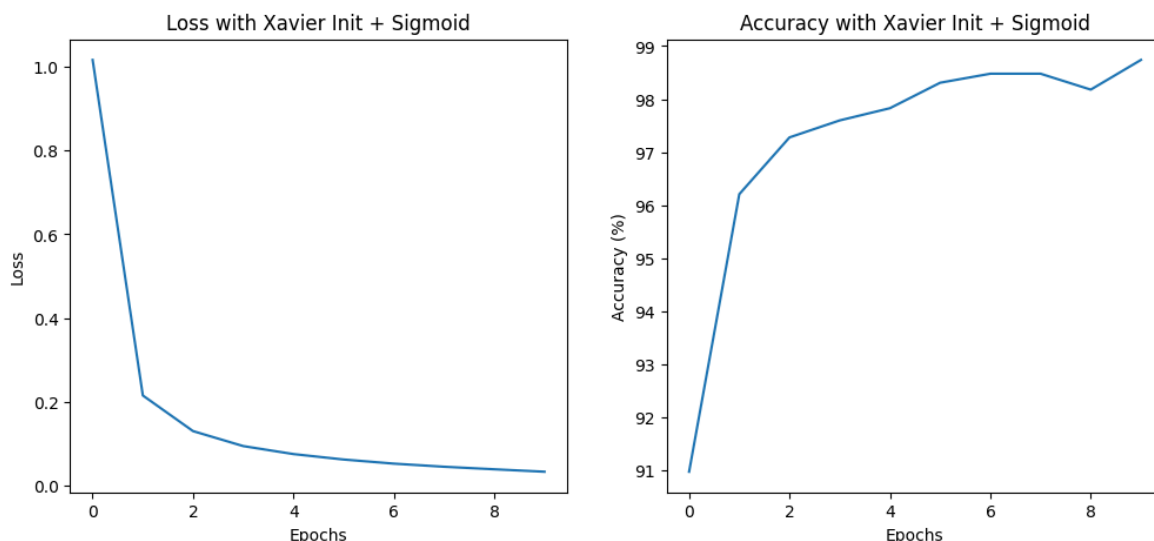
plt.subplot(1, 2, 2)
plt.plot(accuracies, label="Test Accuracy")
plt.title("Accuracy with Xavier Init + Sigmoid")
plt.xlabel("Epochs")
plt.ylabel("Accuracy (%)")

```

```

Out[45]: Text(0, 0.5, 'Accuracy (%)')

```



The preparation of the CNN model when applying the Sigmoid activation function with and without the Xavier initialization shows marginal and yet significant changes in achievements. There is effective learning in both the configurations and model yields high accuracy in either of the cases. Nonetheless, Xavier initialization shows some better results. Namely, it reaches a smaller end loss (0.0329 as compared to 0.0378) and test accuracy (98.74 percent in comparison with 98.48 percent) during the tenth epoch. Although this is a minor gain in a shallow network, it points to the stabilizing property of Xavier initialization in networks using Sigmoid activations.

Sigmoid functions have won accreditation of falling into the affliction of vanishing gradients in the event that starting weights are not well properly highly rated. This is corrected in Xavier initialization by making sure that the variance of activations between layers is not too large or too small so that the backpropagated gradients do not lie outside of a useful range. Consequently, learning becomes more resistant and effective, with improved weight adjustments and excellent chances of avoiding saturation of the neurons. Separately, the model can still be utilized without Xavier although it is caused to be less stable, at least at the initial phase of the training. These distinctions would probably hold truer in more complex networks or train runs that are long in time. In general, the use of Xavier initialization conveys a small, regular profit in the preparation of Sigmoid-take part in the neural systems, in addition, to convergence and generality.

```
In [51]: def test(model):
    model.eval()
    correct = 0
    total = 0
    with torch.no_grad():
        for data, target in test_loader:
            data, target = data.to(device), target.to(device)
            outputs = model(data)
            _, predicted = torch.max(outputs, 1)
            total += target.size(0)
            correct += (predicted == target).sum().item()
    return 100 * correct / total
```

```
In [ ]: def train(model, optimizer, criterion, epochs=10):
    model.train()
```



```

loss_history = []
acc_history = []
grad_history = []

selected_layer = model.classifier[1]

for epoch in range(epochs):
    running_loss = 0.0
    total_grad = 0.0
    batch_count = 0

    for data, target in train_loader:
        data, target = data.to(device), target.to(device)
        optimizer.zero_grad()
        output = model(data)
        loss = criterion(output, target)
        loss.backward()

        grad_norm = selected_layer.weight.grad.norm(2).item()
        total_grad += grad_norm
        batch_count += 1

        optimizer.step()
        running_loss += loss.item()

    avg_loss = running_loss / len(train_loader)
    avg_grad = total_grad / batch_count

    loss_history.append(avg_loss)
    grad_history.append(avg_grad)

    acc = test(model)
    acc_history.append(acc)

    print(f"Epoch [{epoch+1}/{epochs}], Loss: {avg_loss:.4f}, GradNorm: {avg

return loss_history, acc_history, grad_history

```

```

In [53]: model = CNN_Sigmoid().to(device)
model.apply(init_weights)

criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=0.001)

losses, accuracies, gradients = train(model, optimizer, criterion, epochs=10)

```

```

Epoch [1/10], Loss: 0.9555, GradNorm: 0.472602, Test Accuracy: 90.60%
Epoch [2/10], Loss: 0.2176, GradNorm: 0.347899, Test Accuracy: 95.51%
Epoch [3/10], Loss: 0.1321, GradNorm: 0.256994, Test Accuracy: 97.04%
Epoch [4/10], Loss: 0.0955, GradNorm: 0.214352, Test Accuracy: 97.34%
Epoch [5/10], Loss: 0.0759, GradNorm: 0.185656, Test Accuracy: 98.10%
Epoch [6/10], Loss: 0.0652, GradNorm: 0.170750, Test Accuracy: 98.05%
Epoch [7/10], Loss: 0.0547, GradNorm: 0.149977, Test Accuracy: 98.37%
Epoch [8/10], Loss: 0.0478, GradNorm: 0.137167, Test Accuracy: 98.45%
Epoch [9/10], Loss: 0.0412, GradNorm: 0.121058, Test Accuracy: 98.28%
Epoch [10/10], Loss: 0.0359, GradNorm: 0.110138, Test Accuracy: 98.53%

```

```

In [64]: epochs_range = range(1, len(losses) + 1)

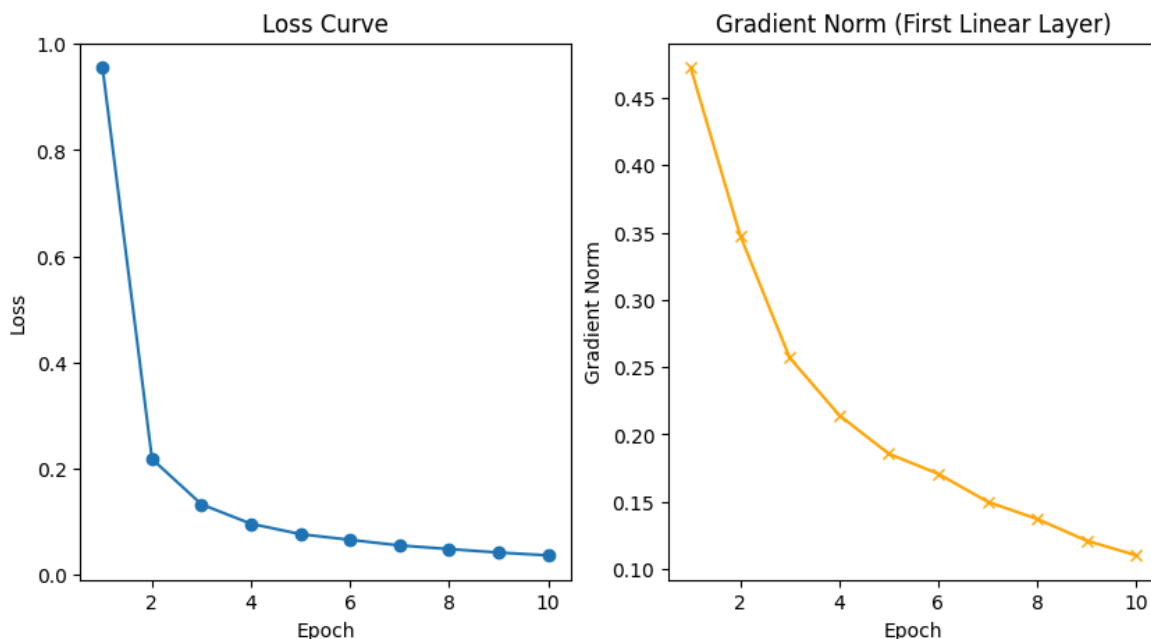
plt.figure(figsize=(10, 5))

```

```
plt.subplot(1, 2, 1)
plt.plot(epochs_range, losses, marker='o', label='Training Loss')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.title('Loss Curve')

plt.subplot(1, 2, 2)
plt.plot(epochs_range, gradients, marker='x', color='orange', label='Gradient L2')
plt.xlabel('Epoch')
plt.ylabel('Gradient Norm')
plt.title('Gradient Norm (First Linear Layer)')
```

Out[64]: Text(0.5, 1.0, 'Gradient Norm (First Linear Layer)')



I have chosen the first linear layer in the classifier to monitor gradient because it is a key element that connects the high-level spatial features resulting by the convolutional layers and the abstract decision-making process by the classifier. This layer is a perfect layer to be used to analyze gradient flow due to the following reasons:

It has a middle depth in the network - it is below the first layers of convolution (which focus on local patterns, e.g., edges), but not as deep as the final layer which can lead to the problem of noisy gradients under propagation of the loss.

The gradients in this layer signify how excellently the learning of transitioning between feature representation and classification is taking place.

It does not reduce to very small gradients or very noisy gradients, and provides a good overall view of training dynamics.

Q2.4

1. ReLU Activation ReLU activation displayed good and stable training benefits. The gradient norm began with a high value of 0.3074 and steadily decline to 0.0702 whereas the loss declined rapidly to 0.0067. This indicates the effectiveness of ReLU in ensuring that the gradients do not become vanishing since it has unbounded

positive value and sparse activation. This enables the error to be back propagated efficiently and increase convergence. Also, the network provided the best result in the test (it reached 99.08%) at epoch 5 only, meaning that ReLU can help to improve the learning process in CNNs on MNIST.

2. Tanh Activation Tanh, unlike ReLU, also displayed resonance by 0.3084 at the beginning of training, but it decreased more slowly overall to 0.1131 at the conclusion of training. The loss was also decreased more slowly-0.9942 to 0.0365. Tanh activations are limited and even and are thus useful in data centering but may cause problem in vanishing gradients at a large magnitude of input values. Because of this, training becomes slower and more gradual. The accuracy after final testing (98.57%) is nearly similar to ReLU although the number of training epochs taken was higher. This gives an indication that although Tanh can be used effectively, it lacks efficiency when it comes to gradient flow and speed of convergence.
3. Sigmoid Activation (Not Xavier Initialization) Application of the Sigmoid activation with no initialisation scheme demonstrated an inferior training behavior. The gradient norm reduced rapidly down to 0.1101 and loss reduction was also high down to 0.0359. The rapid shrinkage of the gradients is the sign of saturation and where the neurons force into the zones of almost zero gradient. This is a well known issue with Sigmoid because it has a narrow active gradient range. Nevertheless, the resulting overall accuracy (98.53) is not that different compared to Tanh, although the stability in training may also be an issue.
4. Sigmoid Activation (With Xavier Initialization) When Xavier initialization was applied with Sigmoid, the training behavior improved notably. The gradient norm started at 0.3084 and decreased more steadily to 0.0456. The loss also dropped more efficiently from 0.0315 to 0.0086, with test accuracy reaching 98.84%. Xavier initialization helps distribute initial weights in a way that maintains variance across layers, effectively mitigating the vanishing gradient problem often seen with Sigmoid. This configuration shows more stable and efficient learning, validating that appropriate initialization can dramatically improve performance with Sigmoid activations.

Conclusion Gradient flow and quick convergence The reason why ReLU is the gradient flow and fastest in convergence is because it is linear and sparse. This is why ReLU is ultimate in deep networks. Tanh has slower gradient decay and slower convergence, but it too attains similar accuracies. Vanishing gradients caused by sigmoid without initialization causes fast decay in the derivatives slowing down the learning. When Xavier initialization is used, Sigmoid is much better and it competes reasonably with Tanh and ReLU. Methods of initialization such as Xavier are vital in maintaining the magnitude of the gradients and as such, subsequent layers of the network can effectively learn, particularly with saturating activations like Sigmoid. ReLU can be seen as the default activation and special care must be taken when using bounded activations.